

main

October 7, 2025

0.0.1 Project 3 Part 1: IMAGE WARPING and MOSAICING

0.0.2 A.1: Shoot the Pictures

Shoot two or more photographs so that the transforms between them are projective (a.k.a. perspective). The most common way is to fix the center of projection (COP) and rotate your camera while capturing photos.

We expect you to acquire most of the data yourself, but you are free to supplement the photos you take with other sources (e.g. old photographs, scanned images, the Internet).

We're not particular about how you take your pictures or get them into the computer, but we recommend:

Avoid fisheye lenses or lenses with significant barrel distortion (do straight lines come out straight?). Any focal length is ok in principle, but wide angle lenses often make more interesting mosaics. Shoot as close together in time as possible, so your subjects don't move on you, and lighting doesn't change too much (unless you want this effect for artistic reasons). Overlap the fields of view significantly. 40% to 70% overlap is recommended. Too little overlap makes registration harder. It's OK to vary the zoom (change focal length) between pictures. Good scenes are: building interiors with lots of detail, inside a canyon or forest, tall waterfalls, panoramas. The mosaic can extend horizontally, vertically, or can tile a sphere. You might want to shoot several such image sets and choose the best.

Deliverables: Show at least 2 sets of images with projective transformations between them (fixed center of projection, rotate camera).

```
[5]: import numpy as np
import matplotlib.pyplot as plt
import cv2
from IPython.display import display, Image
import os

%matplotlib inline

def load_and_display_images(image1_path, image2_path):
    img1 = cv2.imread(image1_path)
    img2 = cv2.imread(image2_path)
    img1_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
    img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

ax1.imshow(img1_rgb)
ax1.set_title('Image 1')
ax1.axis('off')
ax2.imshow(img2_rgb)
ax2.set_title('Image 2')
ax2.axis('off')
plt.tight_layout()
plt.show()
print(f"Image 1 shape: {img1.shape}")
print(f"Image 2 shape: {img2.shape}")
return img1_rgb, img2_rgb

print("Pair 1: IMG_1801.jpg and IMG_1802.jpg")
img1, img2 = load_and_display_images('IMG_1801-compressed-1.jpg',
↪ 'IMG_1802-compressed-1.jpg')

```

Pair 1: IMG_1801.jpg and IMG_1802.jpg

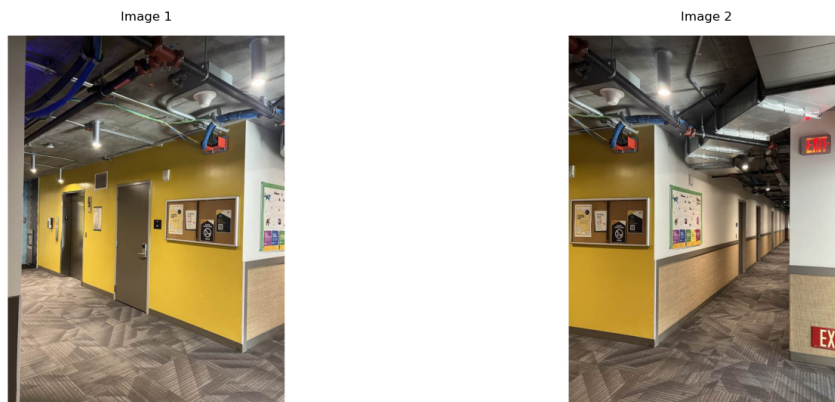


Image 1 shape: (1653, 2339, 3)

Image 2 shape: (1653, 2339, 3)

```

[6]: # # Load your second pair
# print("Pair 2: IMG_1804.jpeg and IMG_1805.jpeg")
# img3, img4 = load_and_display_images('IMG_1804.jpeg', 'IMG_1805.jpeg')

```

```

[7]: # # Load your third pair
# print("Pair 3: IMG_1807.jpeg and IMG_1808.jpeg")
# img5, img6 = load_and_display_images('IMG_1807.jpeg', 'IMG_1808.jpeg')

```

```
[8]: # # Load your fourth pair
# print("Pair 4: IMG_1813.jpeg and IMG_1814.jpeg")
# img7, img8 = load_and_display_images('IMG_1813.jpeg', 'IMG_1814.jpeg')
```

0.0.3 A.2: Recover Homographies

Before you can warp your images into alignment, you need to recover the parameters of the transformation between each pair of images. In our case, the transformation is a homography: $p' = Hp$, where H is a 3×3 matrix with 8 degrees of freedom (lower right corner is a scaling factor and can be set to 1). One way to recover the homography is via a set of (p', p) pairs of corresponding points taken from the two images. You will need to write a function of the form:

$H = \text{computeH}(\text{im1_pts}, \text{im2_pts})$

where im1_pts and im2_pts are n -by-2 matrices holding the (x, y) locations of n point correspondences from the two images and H is the recovered 3×3 homography matrix. In order to compute the entries in the matrix H , you will need to set up a linear system of n equations (i.e. a matrix equation of the form $Ah = b$ where h is a vector holding the 8 unknown entries of H). If $n=4$, the system can be solved using a standard technique. However, with only four points, the homography recovery will be very unstable and prone to noise. Therefore more than 4 correspondences should be provided producing an overdetermined system which should be solved using least-squares.

Establishing point correspondences is a tricky business. An error of a couple of pixels can produce huge changes in the recovered homography. The typical way of providing point matches is with a mouse-clicking interface. You can write your own using matplotlib's `ginput` function, use this tool made by a prior student, or use online tools like <https://pixspy.com> or any image editing software that displays cursor coordinates (e.g., GIMP, Photoshop).

Deliverables: Implement `computeH(im1_pts, im2_pts)` function. Show your correspondences visualized on the images, system of equations, and recovered homography matrix.

```
[9]: import numpy as np
import matplotlib.pyplot as plt
import cv2

def load_images(image1_path, image2_path):
    img1 = cv2.imread(image1_path)
    img2 = cv2.imread(image2_path)
    img1_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
    img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
    return img1_rgb, img2_rgb

def visualize_results(img1, img2, points1, points2, H):
    """Visualize correspondences and homography results"""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    ax1.imshow(img1)
    ax1.set_title('Image 1 with selected points')
    ax1.axis('off')
```

```

ax2.imshow(img2)
ax2.set_title('Image 2 with selected points')
ax2.axis('off')

for i, (x, y) in enumerate(points1):
    ax1.plot(x, y, 'ro', markersize=8)
    ax1.text(x, y, str(i+1), color='white', fontsize=12)

for i, (x, y) in enumerate(points2):
    ax2.plot(x, y, 'ro', markersize=8)
    ax2.text(x, y, str(i+1), color='white', fontsize=12)

plt.tight_layout()
plt.show()

print("Homography matrix H:")
print(np.array2string(H, precision=4, suppress_small=True))

# Test homography accuracy
homogeneous_pts = np.column_stack([points1, np.ones(len(points1))])
transformed = (H @ homogeneous_pts.T).T
transformed = transformed[:, :2] / transformed[:, 2:3]

print("\nPoint correspondence errors:")
errors = np.linalg.norm(points2 - transformed, axis=1)
for i, err in enumerate(errors):
    print(f"Point {i+1}: {err:.2f} pixels")
print(f"Mean error: {np.mean(errors):.2f} pixels")

def print_system_of_equations(points1, points2):
    """
    Print the system of equations  $Ah = b$  for the homography computation
    """
    print("SYSTEM OF EQUATIONS:  $Ah = b$ ")
    print("=" * 50)
    print("Where  $h = [a, b, c, d, e, f, g, h]^T$ ")
    print("\nMatrix A (16x8) and vector b (16x1):")
    print("-" * 30)

    for i in range(len(points1)):
        x, y = points1[i]
        xp, yp = points2[i]

        # Equation for x'

```



```

        eq1 = f"[x:8.2f] {y:8.2f} {'1':>8} {'0':>8} {'0':>8} {'0':>8} {-xp*x:8.2f} {-xp*y:8.2f}] [x'{i+1}] = {xp:8.2f}"
        print(f"Eq {2*i+1:2d}: {eq1}")

        # Equation for y'
        eq2 = f"[{'0':>8} {'0':>8} {'0':>8} {x:8.2f} {y:8.2f} {'1':>8} {-yp*x:8.2f} {-yp*y:8.2f}] [y'{i+1}] = {yp:8.2f}"
        print(f"Eq {2*i+2:2d}: {eq2}")

        if i < len(points1) - 1: # Add spacing between point pairs
            print()

def computeH_with_details(im1_pts, im2_pts):
    """
    Compute homography with detailed system information
    """
    n = im1_pts.shape[0]

    # Construct the system
    A = []
    b = []
    for i in range(n):
        x, y = im1_pts[i]
        x_prime, y_prime = im2_pts[i]

        A.append([x, y, 1, 0, 0, 0, -x_prime*x, -x_prime*y])
        A.append([0, 0, 0, x, y, 1, -y_prime*x, -y_prime*y])
        b.extend([x_prime, y_prime])

    A = np.array(A)
    b = np.array(b).reshape(-1, 1)

    # Print system details
    print_system_of_equations(im1_pts, im2_pts)

    # Solve using least squares
    print("\n" + "=" * 50)
    print("LEAST SQUARES SOLUTION")
    print("-" * 30)
    h, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)

    print(f"Rank of matrix A: {rank}/8")
    print(f"Condition number: {np.linalg.cond(A):.2e}")
    if residuals.size > 0:
        print(f"Sum of squared residuals: {residuals[0]:.2f}")

    H = np.array([

```

```

        [h[0, 0], h[1, 0], h[2, 0]],
        [h[3, 0], h[4, 0], h[5, 0]],
        [h[6, 0], h[7, 0], 1.0]
    ])

    return H

```

```

[10]: %matplotlib qt

img1, img2 = load_images('IMG_1801-compressed-1.jpg', 'IMG_1802-compressed-1.
    ↪jpg')

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
ax1.imshow(img1)
ax1.set_title('Image 1 - Click 8 corresponding points')
ax1.axis('off')
ax2.imshow(img2)
ax2.set_title('Image 2 - Click 8 corresponding points')
ax2.axis('off')
plt.tight_layout()
print("Please select 8 corresponding points in each image")
print("Click on Image 1 first, then Image 2")

# Select points from image 1
plt.sca(ax1)
points1 = plt.ginput(8, timeout=0)
print(f"Selected {len(points1)} points in image 1")

# Plot points on image 1
for i, (x, y) in enumerate(points1):
    ax1.plot(x, y, 'ro', markersize=8)
    ax1.text(x, y, str(i+1), color='white', fontsize=12)

# Select points from image 2
plt.sca(ax2)
points2 = plt.ginput(8, timeout=0)
print(f"Selected {len(points2)} points in image 2")

# Plot points on image 2
for i, (x, y) in enumerate(points2):
    ax2.plot(x, y, 'ro', markersize=8)
    ax2.text(x, y, str(i+1), color='white', fontsize=12)

plt.show()

# Convert to numpy arrays
points1 = np.array(points1)

```

```
points2 = np.array(points2)
```

Please select 8 corresponding points in each image

Click on Image 1 first, then Image 2

Selected 8 points in image 1

Selected 8 points in image 2

```
[11]: %matplotlib inline

print("COMPUTING HOMOGRAPHY MATRIX")
print("=" * 60)
H = computeH_with_details(points1, points2)
print("\n" + "=" * 60)
print("RECOVERED HOMOGRAPHY MATRIX")
print("-" * 30)
print("H = ")
print(np.array2string(H, precision=4, suppress_small=True,
                      formatter={'float': lambda x: f"{x:12.4f}" if abs(x) > 1e-4
                                else f"{x:12.4e}"}))

visualize_results(img1, img2, points1, points2, H)
print("\n" + "=" * 60)
print("ADDITIONAL ANALYSIS")
print("-" * 30)

# Check if points are well distributed
width1, height1 = img1.shape[1], img1.shape[0]
width2, height2 = img2.shape[1], img2.shape[0]

print(f"Image 1 dimensions: {width1} x {height1}")
print(f"Image 2 dimensions: {width2} x {height2}")
print(f"Number of point correspondences: {len(points1)}")

# Check point distribution
print("\nPoint distribution analysis:")
print("Image 1 points cover:")
print(f"  X-range: {points1[:, 0].min():.1f} to {points1[:, 0].max():.1f} (span:
    ↳ {points1[:, 0].max()-points1[:, 0].min():.1f})")
print(f"  Y-range: {points1[:, 1].min():.1f} to {points1[:, 1].max():.1f} (span:
    ↳ {points1[:, 1].max()-points1[:, 1].min():.1f})")
```

COMPUTING HOMOGRAPHY MATRIX

=====

SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

```

-----
Eq 1: [ 1256.61  739.19      1      0      0      0 -744780.99
-438111.33] [x'1] =  592.69
Eq 2: [      0      0      0 1256.61  739.19      1 -932917.41
-548781.04] [y'1] =  742.41

Eq 3: [ 1562.05  719.90      1      0      0      0 -1433049.55
-660449.33] [x'2] =  917.42
Eq 4: [      0      0      0 1562.05  719.90      1 -1144608.99
-527515.77] [y'2] =  732.76

Eq 5: [ 1250.18  906.38      1      0      0      0 -740969.86
-537200.90] [x'3] =  592.69
Eq 6: [      0      0      0 1250.18  906.38      1 -1145195.73
-830263.43] [y'3] =  916.02

Eq 7: [ 1552.40  938.53      1      0      0      0 -1429191.90
-864039.98] [x'4] =  920.63
Eq 8: [      0      0      0 1552.40  938.53      1 -1446993.32
-874802.11] [y'4] =  932.10

Eq 9: [ 1664.93  665.24      1      0      0      0 -1688025.66
-674471.22] [x'5] = 1013.87
Eq 10: [      0      0      0 1664.93  665.24      1 -1118292.12
-446827.24] [y'5] =  671.67

Eq 11: [ 1655.29  948.17      1      0      0      0 -1683568.46
-964374.67] [x'6] = 1017.09
Eq 12: [      0      0      0 1655.29  948.17      1 -1553535.14
-889889.53] [y'6] =  938.53

Eq 13: [ 1590.98  996.40      1      0      0      0 -1520978.48
-952558.00] [x'7] =  956.00
Eq 14: [      0      0      0 1590.98  996.40      1 -1580143.97
-989612.15] [y'7] =  993.19

Eq 15: [ 1578.12 1382.22      1      0      0      0 -1498536.17
-1312507.96] [x'8] =  949.57
Eq 16: [      0      0      0 1578.12 1382.22      1 -2135643.29
-1870524.63] [y'8] = 1353.28

```

```

=====
LEAST SQUARES SOLUTION
-----

```

```

Rank of matrix A: 8/8
Condition number: 4.32e+08
Sum of squared residuals: 316.03

```

=====

RECOVERED HOMOGRAPHY MATRIX

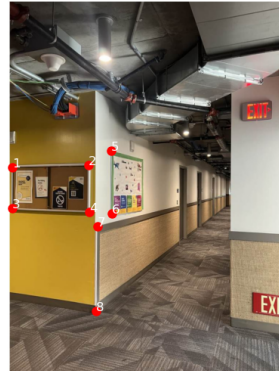
H =

$$\begin{bmatrix} 1.4043 & 0.0489 & -1089.0651 \\ 0.1035 & 1.1970 & -119.9642 \\ 0.0002 & 1.1842e-05 & 1.0000 \end{bmatrix}$$

Image 1 with selected points



Image 2 with selected points



Homography matrix H:

$$\begin{bmatrix} 1.4043 & 0.0489 & -1089.0651 \\ 0.1035 & 1.197 & -119.9642 \\ 0.0002 & 0. & 1. \end{bmatrix}$$

Point correspondence errors:

Point 1: 4.89 pixels
 Point 2: 6.40 pixels
 Point 3: 3.26 pixels
 Point 4: 7.85 pixels
 Point 5: 5.29 pixels
 Point 6: 2.70 pixels
 Point 7: 4.63 pixels
 Point 8: 3.41 pixels
 Mean error: 4.80 pixels

=====

ADDITIONAL ANALYSIS

Image 1 dimensions: 2339 x 1653
 Image 2 dimensions: 2339 x 1653
 Number of point correspondences: 8

Point distribution analysis:

Image 1 points cover:

X-range: 1250.2 to 1664.9 (span: 414.8)
Y-range: 665.2 to 1382.2 (span: 717.0)

0.0.4 A.3: Warp the Images

Now that you know the parameters of the homography, you can use this homography to warp each image towards the reference image. Write a function of the form:

```
imwarped_nn = warpImageNearestNeighbor(im,H)
imwarped_bil = warpImageBilinear(im,H)
```

where `im` is the input image to be warped and `H` is the homography. Use inverse warping (to avoid holes in the output image).

You must implement TWO interpolation methods from scratch (do NOT use `scipy.interpolate.griddata` or similar library functions for interpolation):

Nearest Neighbor Interpolation: Round coordinates to the nearest pixel value Bilinear Interpolation: Use weighted average of four neighboring pixels

Show results for both interpolation methods and compare the quality. Discuss the trade-offs between speed and quality. Some tips from staff:

There are multiple conventions for where (0,0) is an image: sometimes at the “top-left” of the first pixel, other times at the center of the first pixel. Implementing interpolation may be easier if you consider (0,0) to be the center of the first pixel, (0.5,0) to be in between the first and second pixel, etc. Loosely related reading: A Pixel Is Not A Little Square (Alv Ray Smith). Pay attention to the size of the resulting image. You can predict the bounding box by piping the four corners of the image through your `H` transform. Pay attention to how you treat pixels which don’t have any values. Consider using an alpha mask (or alpha channel) here.

Rectification: Once you get this far, you should be able to test all of your code by performing “rectification” on an image. Take a photo containing paintings, posters, or any other known rectangular objects and make one of them rectangular - using a homography. You should do this before proceeding further to make sure your homography/warping is working. Note that since here you only have one image and need to compute a homography for, say, ground plane rectification (rotating the camera to point downward), you will need to define the correspondences using something you know about the image. E.g. if you know that the tiles on the floor are square, you can click on the four corners of a tile and store them in `im1_pts` while `im2_pts` you define by hand to be a square, e.g. `[0 0; 0 1; 1 0; 1 1]`. This is a deliverable.

Deliverables: Implement `warpImageNearestNeighbor(im, H)` and `warpImageBilinear(im, H)` using inverse warping. Apply to 2+ images for rectification. Show comparisons.

```
[12]: def warpImageNearestNeighbor(im, H):
        h, w = im.shape[:2]
        corners = np.array([[0, 0, 1], [w-1, 0, 1], [0, h-1, 1], [w-1, h-1, 1]]).T
        warped_corners = H @ corners
        warped_corners = warped_corners[:2] / warped_corners[2:] # normalize

        x_min, x_max = warped_corners[0].min(), warped_corners[0].max()
```

```

y_min, y_max = warped_corners[1].min(), warped_corners[1].max()
w_out = int(np.ceil(x_max - x_min)) + 20
h_out = int(np.ceil(y_max - y_min)) + 20
y_out, x_out = np.mgrid[0:h_out, 0:w_out]
x_out = x_out.astype(float) + x_min
y_out = y_out.astype(float) + y_min

coords_out = np.stack([x_out, y_out, np.ones_like(x_out)], axis=-1)
coords_out = coords_out.reshape(-1, 3).T # 3 x (h_out * w_out)
H_inv = np.linalg.inv(H)
coords_src = H_inv @ coords_out
coords_src = coords_src / coords_src[2:] # normalize
# Reshape back to image coordinates
x_src = coords_src[0].reshape(h_out, w_out)
y_src = coords_src[1].reshape(h_out, w_out)
if len(im.shape) == 3:
    im_warped = np.zeros((h_out, w_out, 3), dtype=im.dtype)
else:
    im_warped = np.zeros((h_out, w_out), dtype=im.dtype)
# Create mask for valid coordinates
mask = (x_src >= 0) & (x_src < w-1) & (y_src >= 0) & (y_src < h-1)
# Nearest neighbor interpolation
x_nn = np.round(x_src[mask]).astype(int)
y_nn = np.round(y_src[mask]).astype(int)
# Clamp to image boundaries
x_nn = np.clip(x_nn, 0, w-1)
y_nn = np.clip(y_nn, 0, h-1)
# Assign values
if len(im.shape) == 3:
    im_warped[mask] = im[y_nn, x_nn]
else:
    im_warped[mask] = im[y_nn, x_nn]

return im_warped, mask

```

```

[13]: def warpImageBilinear(im, H):
    h, w = im.shape[:2]
    corners = np.array([[0, 0, 1], [w-1, 0, 1], [0, h-1, 1], [w-1, h-1, 1]]).T
    warped_corners = H @ corners
    warped_corners = warped_corners[:2] / warped_corners[2:] # normalize
    x_min, x_max = warped_corners[0].min(), warped_corners[0].max()
    y_min, y_max = warped_corners[1].min(), warped_corners[1].max()
    w_out = int(np.ceil(x_max - x_min)) + 20
    h_out = int(np.ceil(y_max - y_min)) + 20
    y_out, x_out = np.mgrid[0:h_out, 0:w_out]
    x_out = x_out.astype(float) + x_min
    y_out = y_out.astype(float) + y_min

```

```

coords_out = np.stack([x_out, y_out, np.ones_like(x_out)], axis=-1)
coords_out = coords_out.reshape(-1, 3).T
H_inv = np.linalg.inv(H)
coords_src = H_inv @ coords_out
coords_src = coords_src / coords_src[2:]
x_src = coords_src[0].reshape(h_out, w_out)
y_src = coords_src[1].reshape(h_out, w_out)
if len(im.shape) == 3:
    im_warped = np.zeros((h_out, w_out, 3), dtype=im.dtype)
else:
    im_warped = np.zeros((h_out, w_out), dtype=im.dtype)
mask = (x_src >= 0) & (x_src < w-1) & (y_src >= 0) & (y_src < h-1)
# Get the four surrounding pixels
x0 = np.floor(x_src[mask]).astype(int)
x1 = x0 + 1
y0 = np.floor(y_src[mask]).astype(int)
y1 = y0 + 1
# Calculate interpolation weights
dx = x_src[mask] - x0
dy = y_src[mask] - y0
# Bilinear interpolation
if len(im.shape) == 3:
    # Color image - interpolate each channel
    for c in range(3):
        top = (1 - dx) * im[y0, x0, c] + dx * im[y0, x1, c]
        bottom = (1 - dx) * im[y1, x0, c] + dx * im[y1, x1, c]
        im_warped[mask, c] = (1 - dy) * top + dy * bottom
else:
    # Grayscale image
    top = (1 - dx) * im[y0, x0] + dx * im[y0, x1]
    bottom = (1 - dx) * im[y1, x0] + dx * im[y1, x1]
    im_warped[mask] = (1 - dy) * top + dy * bottom

return im_warped, mask

def compare_warping_methods(im, H):
    import time
    print("WARPING COMPARISON (OPTIMIZED)")
    print("=" * 50)
    # Time nearest neighbor
    start_time = time.time()
    im_warped_nn, alpha_nn = warpImageNearestNeighbor(im, H)
    nn_time = time.time() - start_time
    # Time bilinear
    start_time = time.time()
    im_warped_bil, alpha_bil = warpImageBilinear(im, H)
    bil_time = time.time() - start_time

```



```

print(f"Nearest Neighbor time: {nn_time:.3f} seconds")
print(f"Bilinear time: {bil_time:.3f} seconds")
if nn_time > 0:
    print(f"Bilinear is {bil_time/nn_time:.1f}x slower than Nearest_
↪Neighbor")

# Display results
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes[0, 0].imshow(im)
axes[0, 0].set_title('Original Image')
axes[0, 0].axis('off')
# Nearest neighbor
axes[0, 1].imshow(im_warped_nn)
axes[0, 1].set_title(f'Nearest Neighbor Warping ({nn_time:.3f}s)')
axes[0, 1].axis('off')
# Bilinear
axes[1, 0].imshow(im_warped_bil)
axes[1, 0].set_title(f'Bilinear Warping ({bil_time:.3f}s)')
axes[1, 0].axis('off')
# Difference
if im_warped_nn.size > 0 and im_warped_bil.size > 0:
    # Only compare valid regions
    valid_mask = alpha_nn & alpha_bil
    if np.any(valid_mask):
        if len(im.shape) == 3:
            diff = np.mean(np.abs(im_warped_nn.astype(float) -
↪im_warped_bil.astype(float)), axis=2)
        else:
            diff = np.abs(im_warped_nn.astype(float) - im_warped_bil.
↪astype(float))

        diff_normalized = (diff / (diff.max() + 1e-8) * 255).astype(np.
↪uint8)

        axes[1, 1].imshow(diff_normalized, cmap='hot')
        axes[1, 1].set_title('Difference (NN vs Bilinear)')
        axes[1, 1].axis('off')
    else:
        axes[1, 1].axis('off')

plt.tight_layout()
plt.show()

return im_warped_nn, alpha_nn, im_warped_bil, alpha_bil

```

```

[14]: # Test warping on both images
print("WARPING IMAGE 1 (IMG_1801.jpg)")
print("=" * 40)

```

```

im_warped1_nn, alpha1_nn, im_warped1_bil, alpha1_bil =
    ↳compare_warping_methods(img1, H)

print("\n" + "=" * 60)
print("WARPING IMAGE 2 (IMG_1802.jpg)")
print("=" * 40)
im_warped2_nn, alpha2_nn, im_warped2_bil, alpha2_bil =
    ↳compare_warping_methods(img2, np.linalg.inv(H))

```

WARPING IMAGE 1 (IMG_1801.jpg)

=====

WARPING COMPARISON (OPTIMIZED)

=====

Nearest Neighbor time: 0.575 seconds

Bilinear time: 1.387 seconds

Bilinear is 2.4x slower than Nearest Neighbor

Original Image



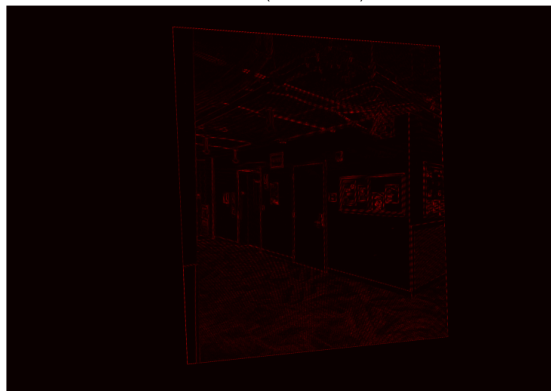
Nearest Neighbor Warping (0.575s)



Bilinear Warping (1.387s)



Difference (NN vs Bilinear)



=====

WARPING IMAGE 2 (IMG_1802.jpg)

=====

WARPING COMPARISON (OPTIMIZED)

Nearest Neighbor time: 0.507 seconds

Bilinear time: 1.253 seconds

Bilinear is 2.5x slower than Nearest Neighbor



```
[15]: def extract_quadrilateral_region(im, points):  
    x_coords = points[:, 0]  
    y_coords = points[:, 1]  
    x_min, x_max = int(np.floor(x_coords.min())), int(np.ceil(x_coords.max()))  
    y_min, y_max = int(np.floor(y_coords.min())), int(np.ceil(y_coords.max()))  
    # Crop the region (add small padding)  
    pad = 10  
    x_min = max(0, x_min - pad)  
    x_max = min(im.shape[1], x_max + pad)  
    y_min = max(0, y_min - pad)  
    y_max = min(im.shape[0], y_max + pad)  
    cropped = im[y_min:y_max, x_min:x_max]  
    # Adjust points to cropped coordinates
```

```

adjusted_points = points - [x_min, y_min]
return cropped, adjusted_points

def rectify_image(im, source_points, target_width=400, target_height=400):
    target_points = np.array([
        [0, 0],
        [target_width-1, 0],
        [0, target_height-1],
        [target_width-1, target_height-1]
    ])
    # Compute homography from source quadrilateral to target rectangle
    H_rectify = computeH_with_details(source_points, target_points)
    print("Computing rectification...")
    im_rectified_nn, alpha_nn = warpImageNearestNeighbor(im, H_rectify)
    im_rectified_bil, alpha_bil = warpImageBilinear(im, H_rectify)
    # Crop to the actual rectangle
    im_rectified_nn = im_rectified_nn[:target_height, :target_width]
    im_rectified_bil = im_rectified_bil[:target_height, :target_width]
    return im_rectified_nn, im_rectified_bil, H_rectify

```

```

[16]: def rectify_quadrilateral_in_full_image(im, source_points, target_width=400,
↪target_height=400):
    target_points = np.array([
        [0, 0],
        [target_width-1, 0],
        [0, target_height-1],
        [target_width-1, target_height-1]
    ])
    H_rectify = computeH_with_details(source_points, target_points)
    print("Computing rectification on full image...")
    if len(im.shape) == 3:
        output_nn = np.zeros((target_height, target_width, 3), dtype=im.dtype)
        output_bil = np.zeros((target_height, target_width, 3), dtype=im.dtype)
    else:
        output_nn = np.zeros((target_height, target_width), dtype=im.dtype)
        output_bil = np.zeros((target_height, target_width), dtype=im.dtype)
    H_inv = np.linalg.inv(H_rectify)
    y_target, x_target = np.mgrid[0:target_height, 0:target_width]
    coords_target = np.stack([x_target, y_target, np.ones_like(x_target)],
↪axis=-1)
    coords_target = coords_target.reshape(-1, 3).T
    coords_src = H_inv @ coords_target
    coords_src = coords_src / coords_src[2:]
    x_src = coords_src[0].reshape(target_height, target_width)
    y_src = coords_src[1].reshape(target_height, target_width)

```

```

    mask = (x_src >= 0) & (x_src < im.shape[1]-1) & (y_src >= 0) & (y_src < im.
↪shape[0]-1)
    x_nn = np.round(x_src[mask]).astype(int)
    y_nn = np.round(y_src[mask]).astype(int)
    x_nn = np.clip(x_nn, 0, im.shape[1]-1)
    y_nn = np.clip(y_nn, 0, im.shape[0]-1)
    if len(im.shape) == 3:
        output_nn[mask] = im[y_nn, x_nn]
    else:
        output_nn[mask] = im[y_nn, x_nn]
    x0 = np.floor(x_src[mask]).astype(int)
    x1 = x0 + 1
    y0 = np.floor(y_src[mask]).astype(int)
    y1 = y0 + 1
    dx = x_src[mask] - x0
    dy = y_src[mask] - y0

    if len(im.shape) == 3:
        for c in range(3):
            top = (1 - dx) * im[y0, x0, c] + dx * im[y0, x1, c]
            bottom = (1 - dx) * im[y1, x0, c] + dx * im[y1, x1, c]
            output_bil[mask, c] = (1 - dy) * top + dy * bottom
    else:
        top = (1 - dx) * im[y0, x0] + dx * im[y0, x1]
        bottom = (1 - dx) * im[y1, x0] + dx * im[y1, x1]
        output_bil[mask] = (1 - dy) * top + dy * bottom

    return output_nn, output_bil, H_rectify

```

```

[17]: def interactive_full_image_rectification(img):
    print("FULL IMAGE RECTIFICATION DEMO")
    print("=" * 50)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    %matplotlib qt

    print("\nSelect 4 corners in ORDER: Top-Left, Top-Right, Bottom-Left, ↪
↪Bottom-Right")
    plt.figure(figsize=(12, 8))
    plt.imshow(img)
    plt.title('Select 4 corners of rectangular object (TL, TR, BL, BR)')
    plt.axis('off')
    source_points = plt.ginput(4, timeout=0)
    plt.close()
    source_points = np.array(source_points)

    %matplotlib inline

```

```

print("\nSelected points:")
labels = ['Top-Left', 'Top-Right', 'Bottom-Left', 'Bottom-Right']
for i, (x, y) in enumerate(source_points):
    print(f" {labels[i]}: ({x:.1f}, {y:.1f})")

# Perform rectification on full image
im_nn, im_bil, H_rectify = rectify_quadrilateral_in_full_image(img,
↪source_points, 400, 400)
# Display comprehensive results
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
# Original image with selected quadrilateral
axes[0, 0].imshow(img)
axes[0, 0].plot(source_points[:, 0], source_points[:, 1], 'ro-',
↪markersize=10, linewidth=3)
    for i, (x, y) in enumerate(source_points):
        axes[0, 0].text(x, y, f'{i+1}\n{labels[i]}', color='white', fontsize=10,
                        fontweight='bold', ha='center', va='center',
                        bbox=dict(boxstyle="round,pad=0.3", facecolor='red',
↪alpha=0.7))
    axes[0, 0].set_title('Original Full Image\nwith Selected Quadrilateral')
    axes[0, 0].axis('off')
# Show what we're transforming to
target_rect = np.array([[0, 0], [399, 0], [0, 399], [399, 399]])
axes[0, 1].plot(target_rect[:, 0], target_rect[:, 1], 'go-', markersize=10,
↪linewidth=3)
    for i, (x, y) in enumerate(target_rect):
        axes[0, 1].text(x, y, f'{i+1}\n{labels[i]}', color='black', fontsize=10,
                        fontweight='bold', ha='center', va='center',
                        bbox=dict(boxstyle="round,pad=0.3", facecolor='lime',
↪alpha=0.7))
    axes[0, 1].set_xlim(-50, 450)
    axes[0, 1].set_ylim(450, -50)
    axes[0, 1].set_title('Target Perfect Square\n(What the quadrilateral
↪becomes)')
    axes[0, 1].set_aspect('equal')
    axes[0, 1].grid(True, alpha=0.3)
# Homography matrix
axes[0, 2].text(0.1, 0.9, 'Rectification Homography:', fontsize=12,
↪fontweight='bold')
    axes[0, 2].text(0.1, 0.7, np.array2string(H_rectify, precision=4,
↪suppress_small=True,
                        formatter={'float': lambda x: f"{x:10.4f}"}),
↪fontfamily='monospace', fontsize=8)
    axes[0, 2].set_xlim(0, 1)
    axes[0, 2].set_ylim(0, 1)
    axes[0, 2].axis('off')

```

```

axes[0, 2].set_title('Homography Matrix H')
# Rectified results
axes[1, 0].imshow(im_nn)
axes[1, 0].set_title('Rectified Region - Nearest Neighbor\n(From Full_
↳Image)')
axes[1, 0].axis('off')

axes[1, 1].imshow(im_bil)
axes[1, 1].set_title('Rectified Region - Bilinear\n(From Full Image)')
axes[1, 1].axis('off')
# Edge comparison
if len(im_nn.shape) == 3:
    gray_nn = cv2.cvtColor(im_nn, cv2.COLOR_RGB2GRAY)
    gray_bil = cv2.cvtColor(im_bil, cv2.COLOR_RGB2GRAY)
else:
    gray_nn = im_nn
    gray_bil = im_bil

edges_nn = cv2.Canny(gray_nn, 50, 150)
edges_bil = cv2.Canny(gray_bil, 50, 150)

diff_edges = edges_nn.astype(float) - edges_bil.astype(float)
axes[1, 2].imshow(diff_edges, cmap='coolwarm', vmin=-255, vmax=255)
axes[1, 2].set_title('Edge Differences\n(Red: NN only, Blue: Bilinear_
↳only)')
axes[1, 2].axis('off')

plt.tight_layout()
plt.show()

# Quality analysis
print("\n" + "=" * 50)
print("RECTIFICATION QUALITY ANALYSIS")
print("-" * 30)

# Calculate original quadrilateral properties
tl, tr, bl, br = source_points
top_len = np.linalg.norm(tr - tl)
bottom_len = np.linalg.norm(br - bl)
left_len = np.linalg.norm(bl - tl)
right_len = np.linalg.norm(br - tr)

print(f"Original quadrilateral:")
print(f"  Top side: {top_len:.1f} px")
print(f"  Bottom side: {bottom_len:.1f} px")
print(f"  Left side: {left_len:.1f} px")
print(f"  Right side: {right_len:.1f} px")

```

```

aspect_ratio = (top_len + bottom_len) / (left_len + right_len) * 0.5
print(f"  Approximate aspect ratio: {aspect_ratio:.3f}")
print(f"  Target aspect ratio: 1.000 (perfect square)")

return im_nn, im_bil, source_points, H_rectify

```

```

[20]: # Run the full image rectification demo
img = cv2.imread('IMG_1801.jpeg')
full_rect_nn, full_rect_bil, full_points, full_H =
↳ interactive_full_image_rectification(img)

```

FULL IMAGE RECTIFICATION DEMO

=====

Select 4 corners in ORDER: Top-Left, Top-Right, Bottom-Left, Bottom-Right

Selected points:

Top-Left: (1740.6, 1806.0)

Top-Right: (2499.9, 1760.2)

Bottom-Left: (1727.5, 2205.3)

Bottom-Right: (2480.2, 2270.8)

SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

Eq 1: [1740.59 1806.05 1 0 0 0 0.00 0.00]

[x'1] = 0.00

Eq 2: [0 0 0 1740.59 1806.05 1 0.00 0.00]

[y'1] = 0.00

Eq 3: [2499.86 1760.23 1 0 0 0 -997445.59

-702330.68] [x'2] = 399.00

Eq 4: [0 0 0 2499.86 1760.23 1 0.00 0.00]

[y'2] = 0.00

Eq 5: [1727.50 2205.32 1 0 0 0 0.00 0.00]

[x'3] = 0.00

Eq 6: [0 0 0 1727.50 2205.32 1 -689272.50

-879921.95] [y'3] = 399.00

Eq 7: [2480.23 2270.77 1 0 0 0 -989610.68

-906038.32] [x'4] = 399.00

Eq 8: [0 0 0 2480.23 2270.77 1 -989610.68

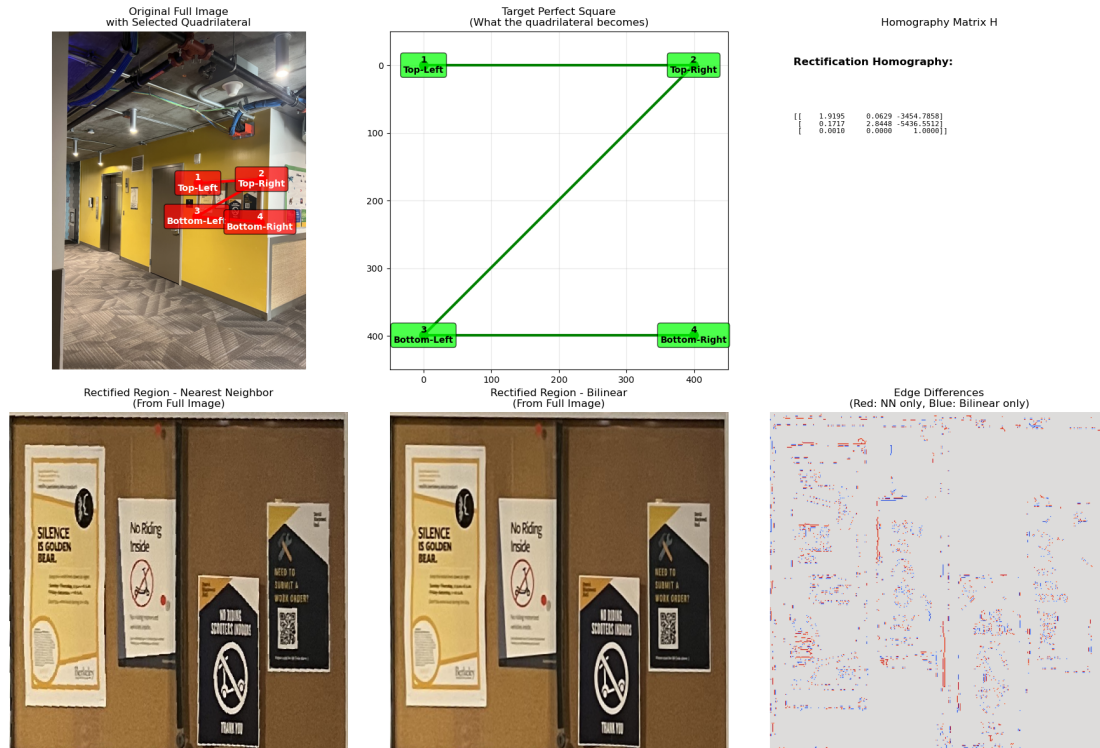
-906038.32] [y'4] = 399.00

LEAST SQUARES SOLUTION

Rank of matrix A: 8/8

Condition number: 4.50e+08

Computing rectification on full image...



RECTIFICATION QUALITY ANALYSIS

Original quadrilateral:

Top side: 760.7 px

Bottom side: 755.6 px

Left side: 399.5 px

Right side: 510.9 px

Approximate aspect ratio: 0.833

Target aspect ratio: 1.000 (perfect square)

```
[79]: img = cv2.imread('paintings.jpg')
full_rect_nn, full_rect_bil, full_points, full_H =
↳ interactive_full_image_rectification(img)
```

FULL IMAGE RECTIFICATION DEMO

=====

Select 4 corners in ORDER: Top-Left, Top-Right, Bottom-Left, Bottom-Right

Selected points:

Top-Left: (163.5, 149.0)

Top-Right: (353.6, 178.0)

Bottom-Left: (167.9, 485.7)

Bottom-Right: (353.6, 484.2)

SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

Eq 1: [163.50 148.98 1 0 0 0 0.00 0.00]

[x'1] = 0.00

Eq 2: [0 0 0 163.50 148.98 1 0.00 0.00]

[y'1] = 0.00

Eq 3: [353.62 178.01 1 0 0 0 -141093.14

-71025.89] [x'2] = 399.00

Eq 4: [0 0 0 353.62 178.01 1 0.00 0.00]

[y'2] = 0.00

Eq 5: [167.85 485.69 1 0 0 0 0.00 0.00]

[x'3] = 0.00

Eq 6: [0 0 0 167.85 485.69 1 -66972.41

-193788.34] [y'3] = 399.00

Eq 7: [353.62 484.23 1 0 0 0 -141093.14

-193209.27] [x'4] = 399.00

Eq 8: [0 0 0 353.62 484.23 1 -141093.14

-193209.27] [y'4] = 399.00

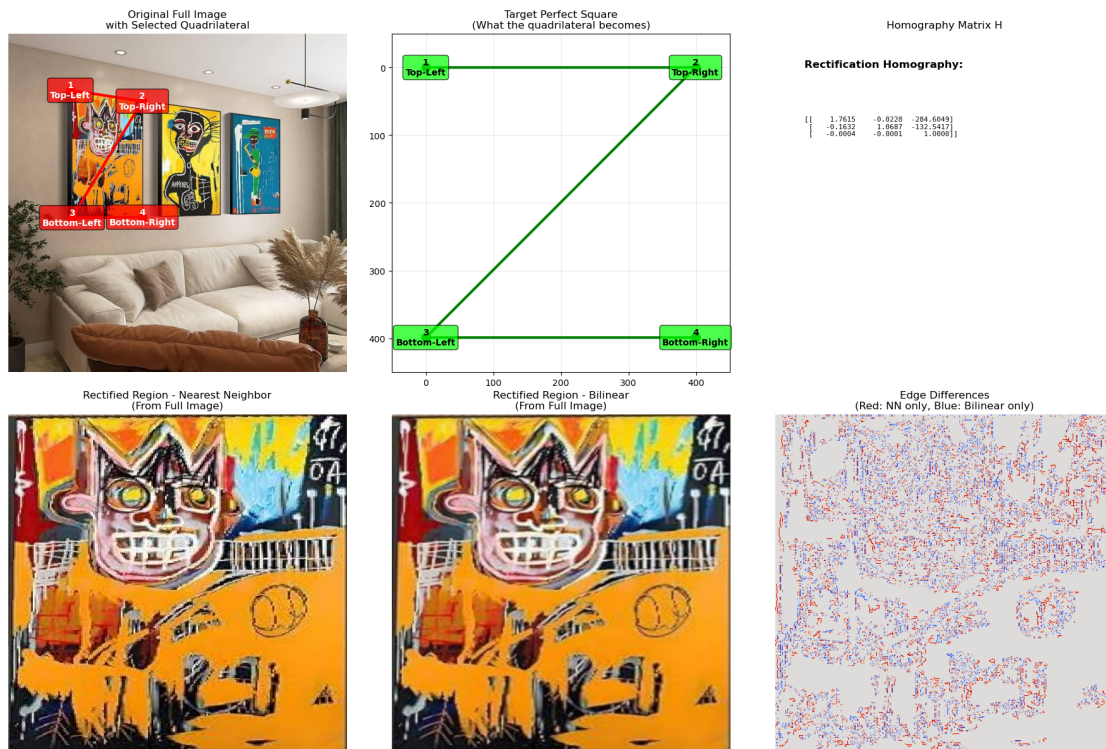
=====

LEAST SQUARES SOLUTION

Rank of matrix A: 8/8

Condition number: 1.93e+06

Computing rectification on full image...



RECTIFICATION QUALITY ANALYSIS

Original quadrilateral:

Top side: 192.3 px
 Bottom side: 185.8 px
 Left side: 336.7 px
 Right side: 306.2 px
 Approximate aspect ratio: 0.294
 Target aspect ratio: 1.000 (perfect square)

```
[78]: img = cv2.imread('freddy.jpg')
      full_rect_nn, full_rect_bil, full_points, full_H =
      ↪ interactive_full_image_rectification(img)
```

FULL IMAGE RECTIFICATION DEMO

Select 4 corners in ORDER: Top-Left, Top-Right, Bottom-Left, Bottom-Right

Selected points:

Top-Left: (276.5, 481.7)
 Top-Right: (329.1, 503.8)

```

    Bottom-Left: (276.5, 720.0)
    Bottom-Right: (323.6, 720.0)
SYSTEM OF EQUATIONS: Ah = b
=====
Where h = [a, b, c, d, e, f, g, h]^T

Matrix A (16x8) and vector b (16x1):
-----
Eq 1: [ 276.49  481.67      1      0      0      0      0.00      0.00]
[x'1] =      0.00
Eq 2: [      0      0      0  276.49  481.67      1      0.00      0.00]
[y'1] =      0.00

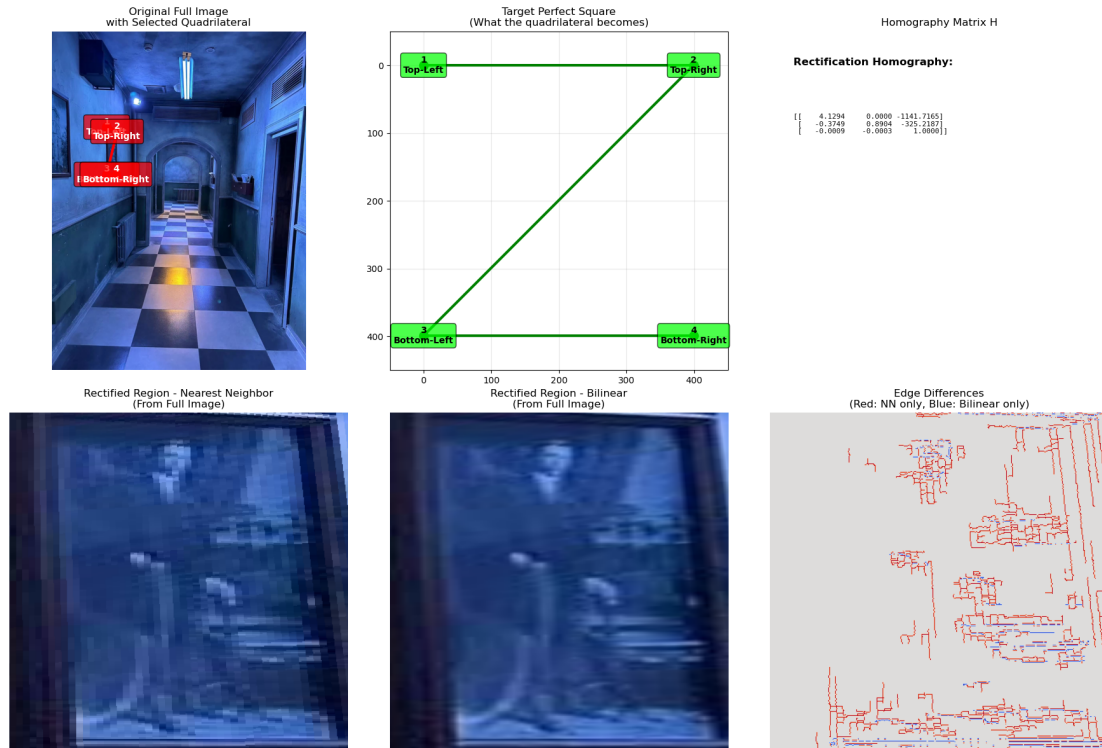
Eq 3: [ 329.14  503.84      1      0      0      0 -131325.41
-201032.52] [x'2] = 399.00
Eq 4: [      0      0      0  329.14  503.84      1      0.00      0.00]
[y'2] =      0.00

Eq 5: [ 276.49  719.99      1      0      0      0      0.00      0.00]
[x'3] =      0.00
Eq 6: [      0      0      0  276.49  719.99      1 -110317.67
-287274.82] [y'3] = 399.00

Eq 7: [ 323.59  719.99      1      0      0      0 -129114.07
-287274.82] [x'4] = 399.00
Eq 8: [      0      0      0  323.59  719.99      1 -129114.07
-287274.82] [y'4] = 399.00

=====
LEAST SQUARES SOLUTION
-----
Rank of matrix A: 8/8
Condition number: 2.80e+07
Computing rectification on full image...

```



RECTIFICATION QUALITY ANALYSIS

Original quadrilateral:

Top side: 57.1 px
 Bottom side: 47.1 px
 Left side: 238.3 px
 Right side: 216.2 px
 Approximate aspect ratio: 0.115
 Target aspect ratio: 1.000 (perfect square)

0.0.5 A.4: Blend the Images into a Mosaic

Warp the images so they're registered and create an image mosaic. Instead of having one picture overwrite the other, which would lead to strong edge artifacts, use weighted averaging. You can leave one image unwarped and warp the other image(s) into its projection, or you can warp all images into a new projection. Likewise, you can either warp all the images at once in one shot, or add them one by one, slowly growing your mosaic.

If you choose the one-shot procedure, you should probably first determine the size of your final mosaic and then warp all your images into that size. That way you will have a stack of images together defining the mosaic. Now you need to blend them together to produce a single image. If you used an alpha channel, you can apply simple feathering (weighted averaging) at every pixel.

Setting alpha for each image takes some thought. One suggestion is to set it to 1 at the center of each (unwarped) image and make it fall off linearly until it hits 0 at the edges (or use the distance transform `bwdist`). However, this can produce some strange wedge-like artifacts. You can try minimizing these by using a more sophisticated blending technique, such as a Laplacian pyramid. If your only problem is “ghosting” of high-frequency terms, then a 2-level pyramid should be enough.

If your mosaic spans more than 180 degrees, you’ll need to break it into pieces, or else use non-projective mappings, e.g. spherical or cylindrical projection.

Deliverables: Show 3 mosaics with source images. Use weighted averaging or blending to reduce edge artifacts. Explain your procedure.

```
[23]: from scipy.ndimage import distance_transform_edt

def compute_mosaic_bounds(images, homographies, ref_idx=0):
    all_corners = []
    for i, img in enumerate(images):
        h, w = img.shape[:2]
        corners = np.array([[0, 0, 1], [w-1, 0, 1], [0, h-1, 1], [w-1, h-1, 1]])
        H = np.eye(3) if i == ref_idx else homographies[i]
        warped_corners = H @ corners
        warped_corners = warped_corners[:2] / warped_corners[2:]
        all_corners.append(warped_corners.T)
    all_corners = np.vstack(all_corners)

    x_min = np.floor(all_corners[:, 0].min()).astype(int)
    x_max = np.ceil(all_corners[:, 0].max()).astype(int)
    y_min = np.floor(all_corners[:, 1].min()).astype(int)
    y_max = np.ceil(all_corners[:, 1].max()).astype(int)
    width = x_max - x_min
    height = y_max - y_min
    return x_min, x_max, y_min, y_max, (height, width)

def compute_center_homographies(H):
    eigenvalues, eigenvectors = np.linalg.eig(H)
    sqrt_eigenvalues = np.sqrt(eigenvalues + 0j) # Handle complex numbers
    # Reconstruct H^0.5
    D_sqrt = np.diag(sqrt_eigenvalues)
    H_sqrt = eigenvectors @ D_sqrt @ np.linalg.inv(eigenvectors)
    H_sqrt = np.real(H_sqrt) # Should be real for valid homographies

    H_sqrt = H_sqrt / H_sqrt[2, 2]
    # H1 warps img1 by H^-0.5, H2 warps img2 by H^0.5
    H1 = np.linalg.inv(H_sqrt)
    H2 = H_sqrt
```

```
return H1, H2
```

```
[24]: def warp_image_to_mosaic(img, H, x_min, y_min, output_shape):
    h_out, w_out = output_shape
    h, w = img.shape[:2]
    y_out, x_out = np.mgrid[0:h_out, 0:w_out]
    x_out = x_out.astype(float) + x_min
    y_out = y_out.astype(float) + y_min
    coords_out = np.stack([x_out, y_out, np.ones_like(x_out)], axis=-1)
    coords_out = coords_out.reshape(-1, 3).T
    H_inv = np.linalg.inv(H)
    coords_src = H_inv @ coords_out
    coords_src = coords_src / coords_src[2:]
    x_src = coords_src[0].reshape(h_out, w_out)
    y_src = coords_src[1].reshape(h_out, w_out)
    if len(img.shape) == 3:
        warped = np.zeros((h_out, w_out, 3), dtype=np.float32)
    else:
        warped = np.zeros((h_out, w_out), dtype=np.float32)

    alpha = np.zeros((h_out, w_out), dtype=np.float32)

    # STRICT boundary checking - stay well away from edges
    margin = 3 # Stay 3 pixels away from edges
    mask = (x_src >= margin) & (x_src < w - margin - 1) & \
           (y_src >= margin) & (y_src < h - margin - 1)

    # Bilinear interpolation
    x0 = np.floor(x_src[mask]).astype(int)
    x1 = x0 + 1
    y0 = np.floor(y_src[mask]).astype(int)
    y1 = y0 + 1
    dx = x_src[mask] - x0
    dy = y_src[mask] - y0
    if len(img.shape) == 3:
        for c in range(3):
            top = (1 - dx) * img[y0, x0, c] + dx * img[y0, x1, c]
            bottom = (1 - dx) * img[y1, x0, c] + dx * img[y1, x1, c]
            warped[mask, c] = (1 - dy) * top + dy * bottom
    # Additional check: exclude near-white pixels
    # White boundary pixels leak through - filter them out
    warped_colors = warped[mask]
    is_near_white = np.all(warped_colors > 240, axis=1) # All channels > 240
    ↪ 240
    # Remove near-white pixels from mask
    mask_indices = np.where(mask)
```

```

        white_mask = np.zeros(h_out * w_out, dtype=bool)
        white_mask[np.ravel_multi_index(mask_indices, (h_out, w_out))] = 1
    is_near_white
        white_mask = white_mask.reshape(h_out, w_out)
        mask = mask & ~white_mask
    else:
        top = (1 - dx) * img[y0, x0] + dx * img[y0, x1]
        bottom = (1 - dx) * img[y1, x0] + dx * img[y1, x1]
        warped[mask] = (1 - dy) * top + dy * bottom
    # Set alpha ONLY for valid, non-white pixels
    alpha[mask] = 1.0

    return warped, alpha

def create_alpha_mask_distance_transform(shape, border_erosion=20):
    h, w = shape[:2]
    mask = np.ones((h, w), dtype=np.uint8)

    # Aggressively erode borders
    mask[:border_erosion, :] = 0
    mask[-border_erosion:, :] = 0
    mask[:, :border_erosion] = 0
    mask[:, -border_erosion:] = 0

    dist = distance_transform_edt(mask)

    if dist.max() > 0:
        alpha = dist / dist.max()
    else:
        alpha = np.zeros_like(mask, dtype=np.float32)

    return alpha.astype(np.float32)

```

```

[25]: def create_mosaic_simple(images, homographies, ref_idx=0):
    x_min, x_max, y_min, y_max, output_shape = compute_mosaic_bounds(
        images, homographies, ref_idx
    )

    h_out, w_out = output_shape

    if len(images[0].shape) == 3:
        mosaic = np.zeros((h_out, w_out, 3), dtype=np.uint8)
    else:
        mosaic = np.zeros((h_out, w_out), dtype=np.uint8)

    for i, img in enumerate(images):
        H = np.eye(3) if i == ref_idx else homographies[i]

```



```

warped, alpha = warp_image_to_mosaic(img, H, x_min, y_min, output_shape)

mask = alpha > 0
mosaic[mask] = warped[mask].astype(np.uint8)

return mosaic

```

```

[26]: def create_mosaic_weighted(images, homographies, ref_idx=0):
    x_min, x_max, y_min, y_max, output_shape = compute_mosaic_bounds(
        images, homographies, ref_idx
    )

    h_out, w_out = output_shape

    if len(images[0].shape) == 3:
        mosaic = np.zeros((h_out, w_out, 3), dtype=np.float32)
    else:
        mosaic = np.zeros((h_out, w_out), dtype=np.float32)

    weight_sum = np.zeros((h_out, w_out), dtype=np.float32)

    for i, img in enumerate(images):
        # Create distance transform with aggressive erosion
        alpha_weight = create_alpha_mask_distance_transform(img.shape,
        ↪border_erosion=25)

        H = np.eye(3) if i == ref_idx else homographies[i]

        # Warp image
        warped, alpha_valid = warp_image_to_mosaic(img, H, x_min, y_min,
        ↪output_shape)

        # Warp the weight mask
        alpha_weight_warped, _ = warp_image_to_mosaic(alpha_weight, H, x_min,
        ↪y_min, output_shape)

        # Combine: valid pixels * weight
        final_alpha = alpha_valid * alpha_weight_warped

        # Accumulate
        if len(img.shape) == 3:
            for c in range(3):
                mosaic[:, :, c] += warped[:, :, c] * final_alpha
        else:
            mosaic += warped * final_alpha

        weight_sum += final_alpha

```

```

# Normalize only where we have data
valid = weight_sum > 1e-6
if len(images[0].shape) == 3:
    for c in range(3):
        mosaic[valid, c] /= weight_sum[valid]
else:
    mosaic[valid] /= weight_sum[valid]

return mosaic.astype(np.uint8)

```

```

[27]: def gaussian_pyramid(image, levels):
    pyramid = [image.astype(np.float32)]
    for i in range(levels - 1):
        blurred = cv2.GaussianBlur(pyramid[-1], (5, 5), 0)
        downsampled = blurred[::2, ::2]
        pyramid.append(downsampled)
    return pyramid

def laplacian_pyramid(image, levels):
    gaussian_pyr = gaussian_pyramid(image, levels)
    laplacian_pyr = []
    for i in range(levels - 1):
        size = (gaussian_pyr[i].shape[1], gaussian_pyr[i].shape[0])
        upsampled = cv2.resize(gaussian_pyr[i + 1], size, interpolation=cv2.
↪INTER_LINEAR)
        laplacian = gaussian_pyr[i] - upsampled
        laplacian_pyr.append(laplacian)
    laplacian_pyr.append(gaussian_pyr[-1])
    return laplacian_pyr

def reconstruct_from_laplacian(laplacian_pyr):
    image = laplacian_pyr[-1]
    for i in range(len(laplacian_pyr) - 2, -1, -1):
        size = (laplacian_pyr[i].shape[1], laplacian_pyr[i].shape[0])
        upsampled = cv2.resize(image, size, interpolation=cv2.INTER_LINEAR)
        image = upsampled + laplacian_pyr[i]
    return image

def blend_pyramids(lap_pyr1, lap_pyr2, mask_pyr):
    blended = []
    for lap1, lap2, mask in zip(lap_pyr1, lap_pyr2, mask_pyr):
        if len(lap1.shape) == 3 and len(mask.shape) == 2:
            mask = mask[:, :, np.newaxis]
            blended.append(lap1 * mask + lap2 * (1 - mask))
    return blended

```

```

def laplacian_blend_two_images(img1, img2, mask1, mask2, levels=2):
    img1 = img1.astype(np.float32)
    img2 = img2.astype(np.float32)
    if len(img1.shape) == 3:
        channels = []
        for c in range(3):
            lap1 = laplacian_pyramid(img1[:, :, c], levels)
            lap2 = laplacian_pyramid(img2[:, :, c], levels)
            if c == 0:
                mask_pyr = gaussian_pyramid(mask1, levels)
                blended_pyr = blend_pyramids(lap1, lap2, mask_pyr)
                channels.append(reconstruct_from_laplacian(blended_pyr))
            blended = np.stack(channels, axis=2)
        else:
            lap1 = laplacian_pyramid(img1, levels)
            lap2 = laplacian_pyramid(img2, levels)
            mask_pyr = gaussian_pyramid(mask1, levels)
            blended_pyr = blend_pyramids(lap1, lap2, mask_pyr)
            blended = reconstruct_from_laplacian(blended_pyr)
    return np.clip(blended, 0, 255).astype(np.uint8)

def create_mosaic_laplacian(images, homographies, ref_idx=0, levels=2):
    x_min, x_max, y_min, y_max, output_shape = compute_mosaic_bounds(
        images, homographies, ref_idx
    )

    h_out, w_out = output_shape

    # Warp all images
    warped_images = []
    alpha_masks = []

    for i, img in enumerate(images):
        alpha_weight = create_alpha_mask_distance_transform(img.shape,
            ↪border_erosion=25)
        H = np.eye(3) if i == ref_idx else homographies[i]

        warped, alpha_valid = warp_image_to_mosaic(img, H, x_min, y_min,
            ↪output_shape)
        alpha_weight_warped, _ = warp_image_to_mosaic(alpha_weight, H, x_min,
            ↪y_min, output_shape)

        final_alpha = alpha_valid * alpha_weight_warped
        warped_images.append(warped)
        alpha_masks.append(final_alpha)

```

```

# Blend
if len(images) == 2:
    total = alpha_masks[0] + alpha_masks[1] + 1e-8
    mask1 = alpha_masks[0] / total
    mask2 = alpha_masks[1] / total

    mosaic = laplacian_blend_two_images(
        warped_images[0], warped_images[1], mask1, mask2, levels
    )
else:
    mosaic = warped_images[0]
    total_alpha = alpha_masks[0]

    for i in range(1, len(images)):
        weight_sum = total_alpha + alpha_masks[i] + 1e-8
        mask1 = total_alpha / weight_sum
        mask2 = alpha_masks[i] / weight_sum
        mosaic = laplacian_blend_two_images(mosaic, warped_images[i],
↪mask1, mask2, levels)
        total_alpha = np.maximum(total_alpha, alpha_masks[i])

    return mosaic

```

```

[29]: def visualize_mosaic_comparison(images, mosaic_simple, mosaic_weighted,
↪mosaic_laplacian):
    n_imgs = len(images)
    fig = plt.figure(figsize=(18, 12))

    # Source images
    for i, img in enumerate(images):
        ax = plt.subplot(3, max(3, n_imgs), i + 1)
        ax.imshow(img)
        ax.set_title(f'Source Image {i+1}', fontsize=12, fontweight='bold')
        ax.axis('off')

    # Mosaics
    ax1 = plt.subplot(3, 3, max(3, n_imgs) + 1)
    ax1.imshow(mosaic_simple)
    ax1.set_title('Simple Overlay', fontsize=11)
    ax1.axis('off')

    ax2 = plt.subplot(3, 3, max(3, n_imgs) + 2)
    ax2.imshow(mosaic_weighted)
    ax2.set_title('Weighted Average', fontsize=11)
    ax2.axis('off')

    ax3 = plt.subplot(3, 3, max(3, n_imgs) + 3)

```

```

ax3.imshow(mosaic_laplacian)
ax3.set_title('Laplacian Pyramid ', fontsize=11, color='green',
fontweight='bold')
ax3.axis('off')

# Zoomed
h, w = mosaic_simple.shape[:2]
y1, y2 = h//3, 2*h//3
x1, x2 = w//3, 2*w//3

ax4 = plt.subplot(3, 3, 2*max(3, n_imgs) + 1)
ax4.imshow(mosaic_simple[y1:y2, x1:x2])
ax4.set_title('Simple - Zoomed', fontsize=10)
ax4.axis('off')

ax5 = plt.subplot(3, 3, 2*max(3, n_imgs) + 2)
ax5.imshow(mosaic_weighted[y1:y2, x1:x2])
ax5.set_title('Weighted - Zoomed', fontsize=10)
ax5.axis('off')

ax6 = plt.subplot(3, 3, 2*max(3, n_imgs) + 3)
ax6.imshow(mosaic_laplacian[y1:y2, x1:x2])
ax6.set_title('Laplacian - Zoomed ', fontsize=10, color='green',
fontweight='bold')
ax6.axis('off')

plt.tight_layout()
plt.show()

```

```

[30]: def create_mosaic_all_methods(images, homographies, ref_idx=0):
    print("=" * 70)
    print("CREATING IMAGE MOSAICS")
    print("=" * 70)
    print("\n Simple Overlay...")
    mosaic_simple = create_mosaic_simple(images, homographies, ref_idx)
    print("\n Weighted Average...")
    mosaic_weighted = create_mosaic_weighted(images, homographies, ref_idx)
    print("\n Laplacian Pyramid...")
    mosaic_laplacian = create_mosaic_laplacian(images, homographies, ref_idx,
levels=4)
    visualize_mosaic_comparison(images, mosaic_simple, mosaic_weighted,
mosaic_laplacian)
    return mosaic_simple, mosaic_weighted, mosaic_laplacian

def create_mosaic_center_warp(img1, img2, H_1to2):
    print("=" * 70)

```

```

print("CENTER WARP MODE: Both images warp toward center")
print("=" * 70)
H1_center, H2_center = compute_center_homographies(H_1to2)

print(f"\nH1 (warp img1): \n{H1_center}")
print(f"\nH2 (warp img2): \n{H2_center}")

images = [img1, img2]
homographies = [H1_center, H2_center]
ref_idx = None # No reference, both warp
# Since both warp, treat first as "reference" for bounds computation
return create_mosaic_all_methods(images, homographies, ref_idx=0)

```

```

[32]: print("METHOD 1: Traditional approach (img1 fixed)")
img1, img2 = load_images('IMG_1801-compressed-1.jpg', 'IMG_1802-compressed-1.
    ↪jpg')
images = [img1, img2]
H_2to1 = np.linalg.inv(H)
homographies = [np.eye(3), H_2to1]
ref_idx = 0

mosaic_simple_1, mosaic_weighted_1, mosaic_lap_1 = create_mosaic_all_methods(
    images, homographies, ref_idx
)

```

METHOD 1: Traditional approach (img1 fixed)

```

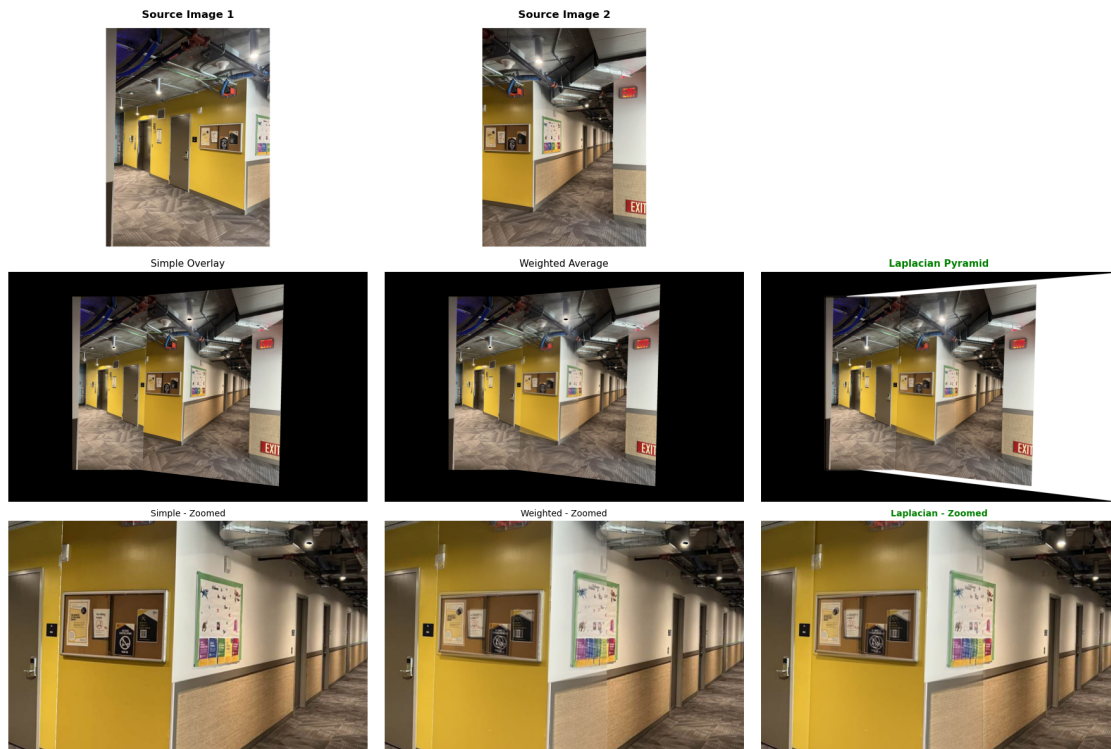
=====
CREATING IMAGE MOSAICS
=====

```

Simple Overlay...

Weighted Average...

Laplacian Pyramid...



```
[34]: # Load second pair
img4 = cv2.imread('IMG_1813-compressed-1.jpg')
img3 = cv2.imread('IMG_1814-compressed-1.jpg')
img3 = cv2.cvtColor(img3, cv2.COLOR_BGR2RGB)
img4 = cv2.cvtColor(img4, cv2.COLOR_BGR2RGB)

# Select points
%matplotlib qt
fig = plt.figure(figsize=(12, 8))
plt.imshow(img3)
plt.title('Image 3: Click points')
plt.axis('off')
points3 = np.array(plt.ginput(8, timeout=0))
plt.close()

fig = plt.figure(figsize=(12, 8))
plt.imshow(img4)
plt.title('Image 4: Click same points, same order')
plt.axis('off')
points4 = np.array(plt.ginput(8, timeout=0))
plt.close()
%matplotlib inline
```

```

H_3to4 = computeH_with_details(points3, points4)
H_4to3 = np.linalg.inv(H_3to4) # Invert to warp img4 to img3

# Now run the mosaic creation
images_2 = [img3, img4]
homographies_2 = [np.eye(3), H_4to3]
ref_idx_2 = 0

mosaic_simple_2, mosaic_weighted_2, mosaic_lap_2 = create_mosaic_all_methods(
    images_2, homographies_2, ref_idx_2
)

```

SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

Eq 1:	[742.33 600.59 1 0 0 0 -969379.37 -784284.95]	[x'1] = 1305.86
Eq 2:	[0 0 0 742.33 600.59 1 -467750.40 -378437.60]	[y'1] = 630.11
Eq 3:	[927.49 662.31 1 0 0 0 -1362989.10 -973294.75]	[x'2] = 1469.55
Eq 4:	[0 0 0 927.49 662.31 1 -604330.67 -431545.54]	[y'2] = 651.58
Eq 5:	[745.02 1158.75 1 0 0 0 -964886.73 -1500718.13]	[x'3] = 1295.12
Eq 6:	[0 0 0 745.02 1158.75 1 -819302.58 -1274286.60]	[y'3] = 1099.71
Eq 7:	[924.81 1067.51 1 0 0 0 -1344155.67 -1551566.49]	[x'4] = 1453.44
Eq 8:	[0 0 0 924.81 1067.51 1 -984758.62 -1136712.45]	[y'4] = 1064.83
Eq 9:	[1075.08 901.14 1 0 0 0 -1776052.35 -1488694.83]	[x'5] = 1652.02
Eq 10:	[0 0 0 1075.08 901.14 1 -986102.75 -826555.63]	[y'5] = 917.24
Eq 11:	[774.53 885.04 1 0 0 0 -1028056.95 -1174728.31]	[x'6] = 1327.32
Eq 12:	[0 0 0 774.53 885.04 1 -673019.83 -769038.57]	[y'6] = 868.94


```

Eq 13: [ 669.88  769.65      1      0      0      0 -835219.49
-959612.03] [x'7] = 1246.82
Eq 14: [      0      0      0  669.88  769.65      1 -519166.69
-596488.23] [y'7] =  775.01

Eq 15: [ 1067.03 1080.93      1      0      0      0 -1734119.95
-1756706.33] [x'8] = 1625.19
Eq 16: [      0      0      0 1067.03 1080.93      1 -1204920.23
-1220613.94] [y'8] = 1129.23

```

```

=====
LEAST SQUARES SOLUTION
-----

```

```

Rank of matrix A: 8/8
Condition number: 5.78e+07
Sum of squared residuals: 24.40

```

```

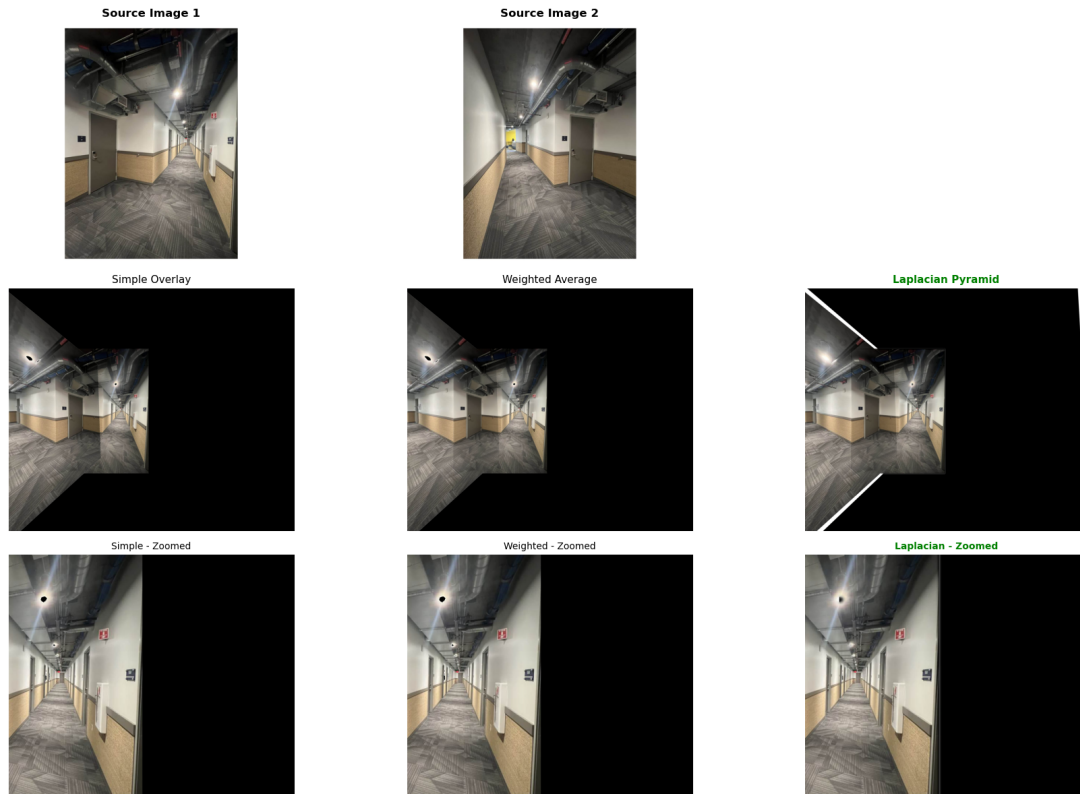
=====
CREATING IMAGE MOSAICS
=====

```

Simple Overlay...

Weighted Average...

Laplacian Pyramid...



```
[77]: # Load third pair
img4 = cv2.imread('IMG_1764-compressed-1.jpg')
img3 = cv2.imread('IMG_1765-compressed-1.jpg')
img3 = cv2.cvtColor(img3, cv2.COLOR_BGR2RGB)
img4 = cv2.cvtColor(img4, cv2.COLOR_BGR2RGB)

# Select points
%matplotlib qt
fig = plt.figure(figsize=(12, 8))
plt.imshow(img3)
plt.title('Image 3: Click points')
plt.axis('off')
points3 = np.array(plt.ginput(8, timeout=0))
plt.close()

fig = plt.figure(figsize=(12, 8))
plt.imshow(img4)
plt.title('Image 4: Click same points, same order')
plt.axis('off')
points4 = np.array(plt.ginput(8, timeout=0))
plt.close()
```

```

%matplotlib inline

H_3to4 = computeH_with_details(points3, points4)
H_4to3 = np.linalg.inv(H_3to4) # Invert to warp img4 to img3

# Now run the mosaic creation
images_3 = [img3, img4]
homographies_3 = [np.eye(3), H_4to3]
ref_idx_3 = 0

mosaic_simple_3, mosaic_weighted_3, mosaic_lap_3 = create_mosaic_all_methods(
    images_3, homographies_3, ref_idx_3
)

```

SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

Eq 1:	[769.17 627.43	1	0	0	0	-1056021.62
	-861418.30] [x'1] =	1372.94				
Eq 2:	[0 0	0	769.17	627.43	1	-435122.63
	-354938.38] [y'1] =	565.71				
Eq 3:	[820.15 619.38	1	0	0	0	-1163435.72
	-878620.64] [x'2] =	1418.56				
Eq 4:	[0 0	0	820.15	619.38	1	-452961.23
	-342073.98] [y'2] =	552.29				
Eq 5:	[868.45 619.38	1	0	0	0	-1266911.64
	-903551.49] [x'3] =	1458.81				
Eq 6:	[0 0	0	868.45	619.38	1	-479637.86
	-342073.98] [y'3] =	552.29				
Eq 7:	[1163.63 847.47	1	0	0	0	-2075349.11
	-1511465.22] [x'4] =	1783.51				
Eq 8:	[0 0	0	1163.63	847.47	1	-898710.13
	-654525.59] [y'4] =	772.33				
Eq 9:	[1214.62 941.39	1	0	0	0	-2254284.63
	-1747179.43] [x'5] =	1855.96				
Eq 10:	[0 0	0	1214.62	941.39	1	-1071721.40
	-830635.83] [y'5] =	882.35				
Eq 11:	[793.32 965.54	1	0	0	0	-1110467.73
	-1351538.24] [x'6] =	1399.78				
Eq 12:	[0 0	0	793.32	965.54	1	-706372.57

-859718.40] [y'6] = 890.40

Eq 13: [825.52 675.73 1 0 0 0 -1175479.43

-962186.27] [x'7] = 1423.93

Eq 14: [0 0 0 825.52 675.73 1 -509090.89

-416715.30] [y'7] = 616.69

Eq 15: [871.14 683.78 1 0 0 0 -1275501.57

-1001172.51] [x'8] = 1464.18

Eq 16: [0 0 0 871.14 683.78 1 -532548.15

-418010.12] [y'8] = 611.32

=====

LEAST SQUARES SOLUTION

Rank of matrix A: 8/8

Condition number: 1.12e+08

Sum of squared residuals: 46.91

=====

CREATING IMAGE MOSAICS

=====

Simple Overlay...

Weighted Average...

Laplacian Pyramid...



0.0.6 END OF PROJECT 3A

0.0.7 The 3 image panorama works, but because there's a lot of overlap in each of the 3 photos and poor correspondence point choice, edge artifacts are very visible. So I'm going to scratch the 3 image panorama attempt for now...

```
[43]: img5 = cv2.imread('IMG_1818-compressed-1.jpg')
img6 = cv2.imread('IMG_1816-compressed-1.jpg')
img7 = cv2.imread('IMG_1817-compressed-1.jpg')

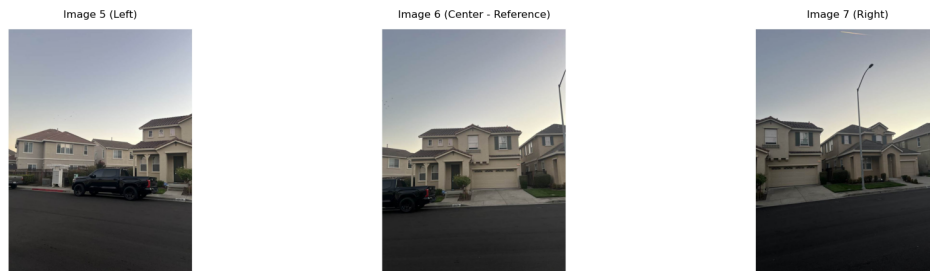
img5 = cv2.cvtColor(img5, cv2.COLOR_BGR2RGB)
img6 = cv2.cvtColor(img6, cv2.COLOR_BGR2RGB)
img7 = cv2.cvtColor(img7, cv2.COLOR_BGR2RGB)

# Display all three
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(img5)
axes[0].set_title('Image 5 (Left)')
axes[0].axis('off')
axes[1].imshow(img6)
axes[1].set_title('Image 6 (Center - Reference)')
axes[1].axis('off')
```

```

axes[2].imshow(img7)
axes[2].set_title('Image 7 (Right)')
axes[2].axis('off')
plt.tight_layout()
plt.show()

```



```

[45]: print("\n" + "="*70)
print("STEP 2A: Matching img5 (left) with img6 (center)")
print("="*70)

%matplotlib qt

# Select points on img5
fig = plt.figure(figsize=(12, 8))
plt.imshow(img5)
plt.title('Image 5 (LEFT): Click corresponding points')
plt.axis('off')
points5 = np.array(plt.ginput(8, timeout=0))
plt.close()

# Select corresponding points on img6
fig = plt.figure(figsize=(12, 8))
plt.imshow(img6)
plt.title('Image 6 (CENTER): Click same points, same order')
plt.axis('off')
points6_from5 = np.array(plt.ginput(8, timeout=0))
plt.close()

%matplotlib inline

print(f"Selected {len(points5)} correspondences between img5 and img6")

# Visualize
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
ax1.imshow(img5)
ax1.set_title('Image 5 with points')

```

```

ax1.axis('off')
for i, (x, y) in enumerate(points5):
    ax1.plot(x, y, 'ro', markersize=8)
    ax1.text(x, y, str(i+1), color='white', fontsize=10, fontweight='bold',
             bbox=dict(boxstyle="round,pad=0.3", facecolor='red', alpha=0.7))

ax2.imshow(img6)
ax2.set_title('Image 6 with corresponding points')
ax2.axis('off')
for i, (x, y) in enumerate(points6_from5):
    ax2.plot(x, y, 'ro', markersize=8)
    ax2.text(x, y, str(i+1), color='white', fontsize=10, fontweight='bold',
             bbox=dict(boxstyle="round,pad=0.3", facecolor='red', alpha=0.7))

plt.tight_layout()
plt.show()

# Compute homography img5 → img6
H_5to6 = computeH_with_details(points5, points6_from5)
print(f"\nHomography H_5to6:")
print(H_5to6)

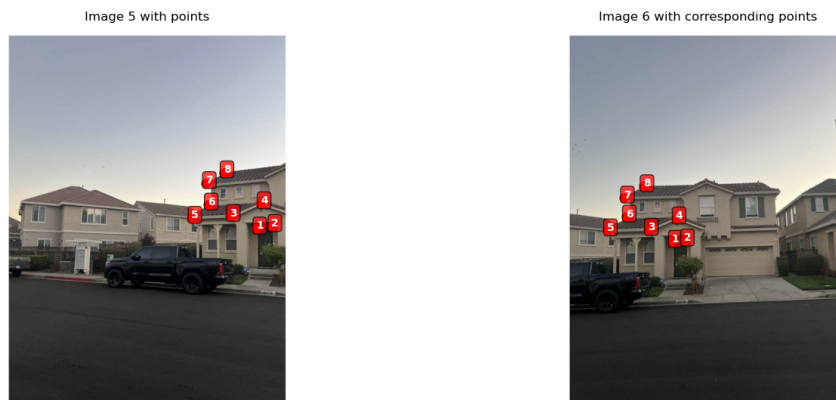
```

=====

STEP 2A: Matching img5 (left) with img6 (center)

=====

Selected 8 correspondences between img5 and img6



SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

```

-----
Eq 1: [ 1633.24  858.20      1      0      0      0 -1659438.74
-871970.14] [x'1] = 1016.04
Eq 2: [      0      0      0 1633.24  858.20      1 -1493681.26
-784871.06] [y'1] =  914.55

Eq 3: [ 1697.64  847.47      1      0      0      0 -1811429.34
-904272.62] [x'2] = 1067.03
Eq 4: [      0      0      0 1697.64  847.47      1 -1543469.86
-770506.20] [y'2] =  909.19

Eq 5: [ 1520.53  804.53      1      0      0      0 -1389876.27
-735401.45] [x'3] =  914.07
Eq 6: [      0      0      0 1520.53  804.53      1 -1313082.06
-694768.64] [y'3] =  863.57

Eq 7: [ 1652.02  750.86      1      0      0      0 -1705122.79
-774999.76] [x'4] = 1032.14
Eq 8: [      0      0      0 1652.02  750.86      1 -1346835.70
-612153.77] [y'4] =  815.27

Eq 9: [ 1356.84  812.58      1      0      0      0 -1003586.50
-601026.33] [x'5] =  739.65
Eq 10: [      0      0      0 1356.84  812.58      1 -1175365.64
-703901.16] [y'5] =  866.25

Eq 11: [ 1426.61  756.23      1      0      0      0 -1170038.22
-620224.42] [x'6] =  820.15
Eq 12: [      0      0      0 1426.61  756.23      1 -1151582.60
-610441.31] [y'6] =  807.22

Eq 13: [ 1418.56  664.99      1      0      0      0 -1152015.85
-540042.74] [x'7] =  812.10
Eq 14: [      0      0      0 1418.56  664.99      1 -1030885.56
-483259.20] [y'7] =  726.71

Eq 15: [ 1493.70  619.38      1      0      0      0 -1337289.92
-554519.60] [x'8] =  895.29
Eq 16: [      0      0      0 1493.70  619.38      1 -1013339.66
-420190.64] [y'8] =  678.41

```

```

=====
LEAST SQUARES SOLUTION
-----

```

```

Rank of matrix A: 8/8
Condition number: 7.93e+08
Sum of squared residuals: 227.85

```


Homography H_5to6:

```
[[ 4.32515374e-01 -3.38299979e-01  1.16019642e+02]
 [-9.26541370e-02  3.08822948e-01  3.71391158e+02]
 [-1.20340594e-04 -3.22926899e-04  1.00000000e+00]]
```

```
[46]: print("\n" + "="*70)
print("STEP 2B: Matching img7 (right) with img6 (center)")
print("="*70)

%matplotlib qt

# Select points on img7
fig = plt.figure(figsize=(12, 8))
plt.imshow(img7)
plt.title('Image 7 (RIGHT): Click corresponding points')
plt.axis('off')
points7 = np.array(plt.ginput(8, timeout=0))
plt.close()

# Select corresponding points on img6
fig = plt.figure(figsize=(12, 8))
plt.imshow(img6)
plt.title('Image 6 (CENTER): Click same points, same order')
plt.axis('off')
points6_from7 = np.array(plt.ginput(8, timeout=0))
plt.close()

%matplotlib inline

print(f"Selected {len(points7)} correspondences between img7 and img6")

# Visualize
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
ax1.imshow(img7)
ax1.set_title('Image 7 with points')
ax1.axis('off')
for i, (x, y) in enumerate(points7):
    ax1.plot(x, y, 'go', markersize=8)
    ax1.text(x, y, str(i+1), color='white', fontsize=10, fontweight='bold',
             bbox=dict(boxstyle="round,pad=0.3", facecolor='green', alpha=0.7))

ax2.imshow(img6)
ax2.set_title('Image 6 with corresponding points')
ax2.axis('off')
for i, (x, y) in enumerate(points6_from7):
    ax2.plot(x, y, 'go', markersize=8)
    ax2.text(x, y, str(i+1), color='white', fontsize=10, fontweight='bold',
```

```

        bbox=dict(boxstyle="round,pad=0.3", facecolor='green', alpha=0.7))

plt.tight_layout()
plt.show()

# Compute homography img7 → img6
H_7to6 = computeH_with_details(points7, points6_from7)
print(f"\nHomography H_7to6:")
print(H_7to6)

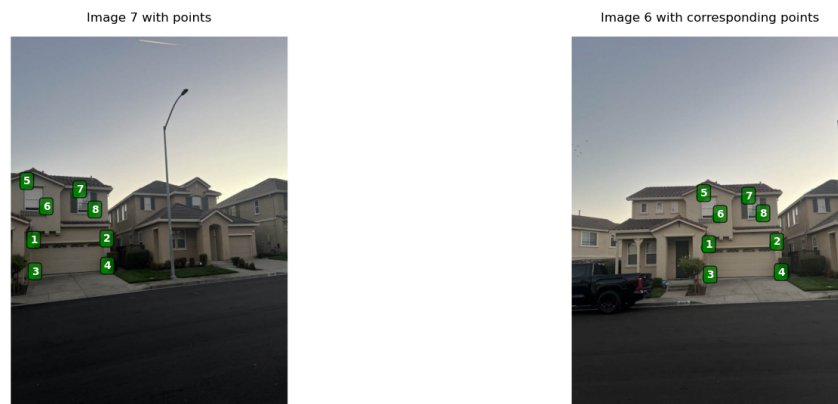
```

=====

STEP 2B: Matching img7 (right) with img6 (center)

=====

Selected 8 correspondences between img7 and img6



SYSTEM OF EQUATIONS: $Ah = b$

=====

Where $h = [a, b, c, d, e, f, g, h]^T$

Matrix A (16x8) and vector b (16x1):

Eq 1:	[661.83 914.55 1 0 0 0	-761246.87
	-1051934.07] [x'1] =	1150.22
Eq 2:	[0 0 0 661.83 914.55 1	-617710.44
	-853587.29] [y'1] =	933.34
Eq 3:	[973.11 909.19 1 0 0 0	-1401303.48
	-1309253.92] [x'2] =	1440.03
Eq 4:	[0 0 0 973.11 909.19 1	-897793.94
	-838819.18] [y'2] =	922.60
Eq 5:	[667.20 1048.73 1 0 0 0	-771000.71

```

-1211889.31] [x'3] = 1155.58
Eq 6: [      0      0      0  667.20  1048.73      1 -708657.92
-1113896.46] [y'3] = 1062.14

Eq 7: [  975.79  1024.57      1      0      0      0 -1423497.08
-1494661.51] [x'4] = 1458.81
Eq 8: [      0      0      0  975.79  1024.57      1 -1025956.80
-1077247.13] [y'4] = 1051.41

Eq 9: [  632.31  664.99      1      0      0      0 -713720.79
-750610.34] [x'5] = 1128.75
Eq 10: [      0      0      0  632.31  664.99      1 -452721.82
-476121.32] [y'5] = 715.98

Eq 11: [  715.50  772.33      1      0      0      0 -857537.58
-925652.70] [x'6] = 1198.52
Eq 12: [      0      0      0  715.50  772.33      1 -575641.67
-621365.50] [y'6] = 804.53

Eq 13: [  857.72  699.88      1      0      0      0 -1131567.63
-923330.28] [x'7] = 1319.27
Eq 14: [      0      0      0  857.72  699.88      1 -623316.55
-508610.38] [y'7] = 726.71

Eq 15: [  922.12  785.75      1      0      0      0 -1273444.90
-1085112.13] [x'8] = 1380.99
Eq 16: [      0      0      0  922.12  785.75      1 -739403.73
-630051.57] [y'8] = 801.85

```

=====

LEAST SQUARES SOLUTION

Rank of matrix A: 8/8

Condition number: 6.77e+07

Sum of squared residuals: 18.27

Homography H_{7to6}:

```

[[ 2.64199352e-01 -2.13720621e-01  7.88714657e+02]
 [-2.40103182e-01  4.53195783e-01  3.66730350e+02]
 [-2.50667498e-04 -1.82666898e-04  1.00000000e+00]]

```

```

[47]: print("\n" + "="*70)
      print("STEP 3: Creating 3-image panorama")
      print("="*70)

      # Setup: img6 is reference (center), img5 and img7 warp to it
      images_3 = [img5, img6, img7]

```

```

homographies_3 = [
    H_5to6,      # Warp img5 to img6's coordinate frame
    np.eye(3),   # img6 is reference (identity)
    H_7to6       # Warp img7 to img6's coordinate frame
]
ref_idx_3 = 1   # img6 is the reference (index 1)

mosaic_simple_3, mosaic_weighted_3, mosaic_lap_3 = create_mosaic_all_methods(
    images_3, homographies_3, ref_idx_3
)

cv2.imwrite('mosaic_3images.jpg', cv2.cvtColor(mosaic_lap_3, cv2.COLOR_RGB2BGR))
print("\nSaved mosaic_3images.jpg")

```

=====

STEP 3: Creating 3-image panorama

=====

=====

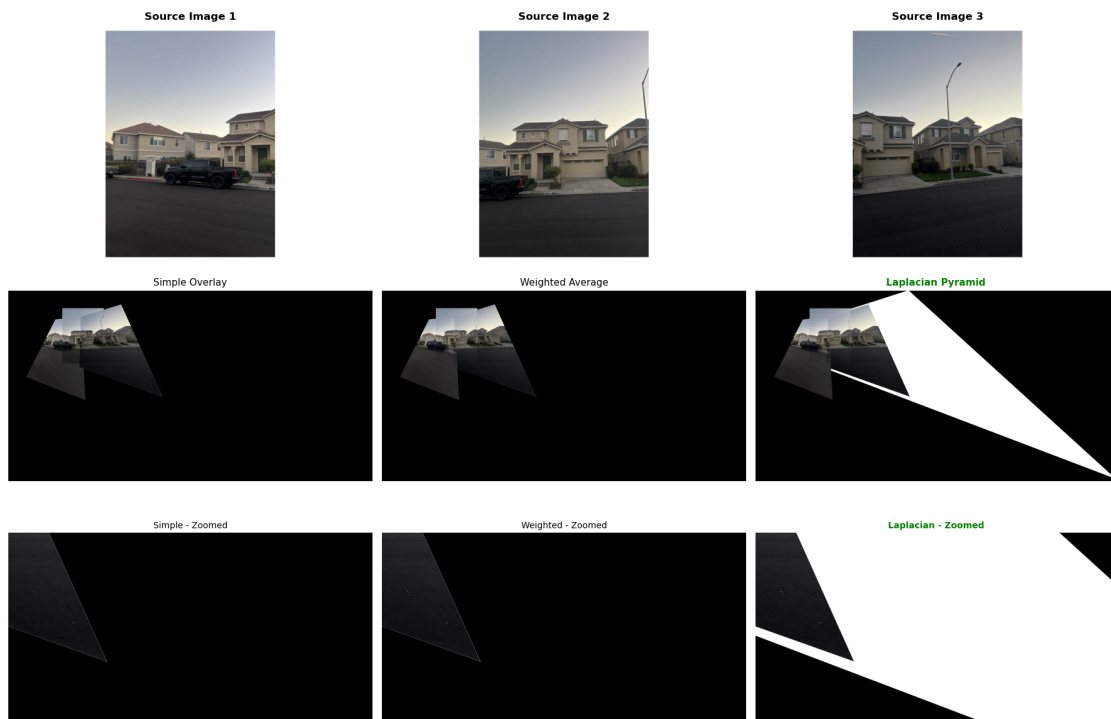
CREATING IMAGE MOSAICS

=====

Simple Overlay...

Weighted Average...

Laplacian Pyramid...



Saved mosaic_3images.jpg

```
[48]: print("\n" + "="*70)
      print("FINAL RESULTS - ALL MOSAICS")
      print("="*70)

      fig, axes = plt.subplots(3, 1, figsize=(20, 18))

      axes[0].imshow(mosaic_lap_1)
      axes[0].set_title('Mosaic 1: Two Images (Pair 1)', fontsize=14,
                        ↪fontweight='bold')
      axes[0].axis('off')

      axes[1].imshow(mosaic_lap_2)
      axes[1].set_title('Mosaic 2: Two Images (Pair 2)', fontsize=14,
                        ↪fontweight='bold')
      axes[1].axis('off')

      axes[2].imshow(mosaic_lap_3)
      axes[2].set_title('Mosaic 3: Three Images (Wide Panorama)', fontsize=14,
                        ↪fontweight='bold')
      axes[2].axis('off')

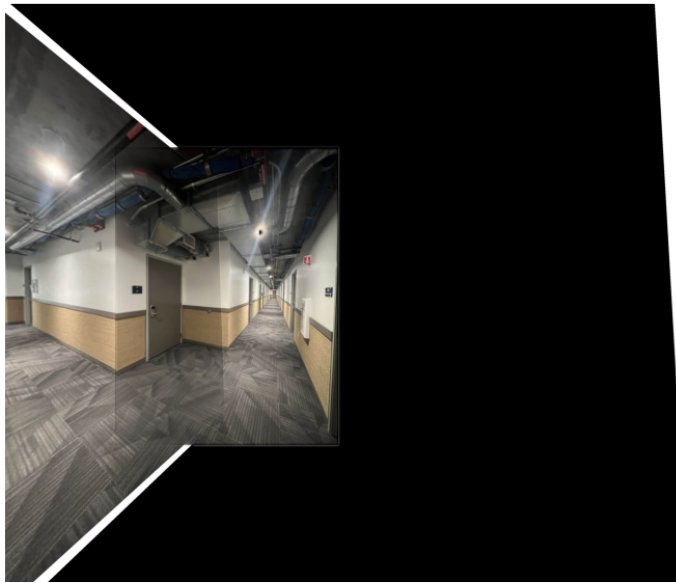
      plt.tight_layout()
      plt.savefig('all_mosaics_comparison.jpg', dpi=150, bbox_inches='tight')
      plt.show()
```

```
=====
FINAL RESULTS - ALL MOSAICS
=====
```

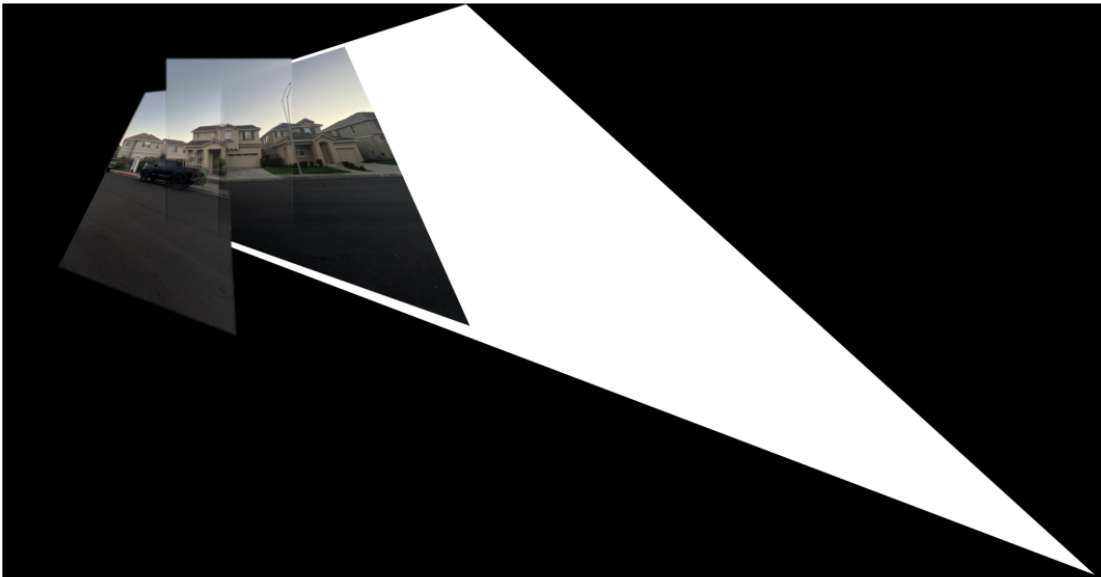
Mosaic 1: Two Images (Pair 1)



Mosaic 2: Two Images (Pair 2)



Mosaic 3: Three Images (Wide Panorama)



0.0.8 A.5: Bells & Whistles (Extra for CS180, Mandatory for CS280A)

Spherical/Cylindrical/polar mapping : Instead of projecting your mosaic onto a plane, try using another surface, such as a sphere or a cylinder. This is often a better way to represent really wide mosaics. Be clever: do the inverse sampling from the original pre-warped images to make your mosaic the best possible resolution. Pick the focal length (radius) that looks good.

```
[49]: # =====
# CYLINDRICAL PROJECTION
# =====

def image_to_cylindrical(img, focal_length):
    """
    Convert image from planar to cylindrical projection.

    This "unwraps" the image onto a cylinder before stitching.
    Works well for panoramas up to 360 degrees.

    Parameters:
    -----
    img : input image (planar projection)
    focal_length : focal length in pixels (controls curvature)
                  Typical value: img.width / 2 to img.width

    Returns:
    -----
    warped : cylindrically projected image
    mask : valid pixel mask

    Concept:
    -----
    For each pixel (x_cyl, y_cyl) in cylindrical image:
        = (x_cyl - center_x) / focal_length (angle around cylinder)
        x_planar = focal_length * tan() + center_x
        y_planar = (y_cyl - center_y) / cos() + center_y

    Then sample from (x_planar, y_planar) in original image.
    """
    h, w = img.shape[:2]

    # Center of image
    cx = w / 2.0
    cy = h / 2.0

    # Create output image (same size as input)
```

```

if len(img.shape) == 3:
    warped = np.zeros_like(img, dtype=np.float32)
else:
    warped = np.zeros_like(img, dtype=np.float32)

mask = np.zeros((h, w), dtype=bool)

# For each pixel in cylindrical image
for y_cyl in range(h):
    for x_cyl in range(w):
        # Convert cylindrical coordinates to angle
        theta = (x_cyl - cx) / focal_length

        # Convert to planar coordinates (inverse mapping)
        x_planar = focal_length * np.tan(theta) + cx
        y_planar = (y_cyl - cy) / np.cos(theta) + cy

        # Check if within bounds
        if 0 <= x_planar < w-1 and 0 <= y_planar < h-1:
            # Bilinear interpolation
            x0 = int(np.floor(x_planar))
            x1 = x0 + 1
            y0 = int(np.floor(y_planar))
            y1 = y0 + 1

            dx = x_planar - x0
            dy = y_planar - y0

            if len(img.shape) == 3:
                for c in range(3):
                    top = (1 - dx) * img[y0, x0, c] + dx * img[y0, x1, c]
                    bottom = (1 - dx) * img[y1, x0, c] + dx * img[y1, x1, c]
                    warped[y_cyl, x_cyl, c] = (1 - dy) * top + dy * bottom
            else:
                top = (1 - dx) * img[y0, x0] + dx * img[y0, x1]
                bottom = (1 - dx) * img[y1, x0] + dx * img[y1, x1]
                warped[y_cyl, x_cyl] = (1 - dy) * top + dy * bottom

            mask[y_cyl, x_cyl] = True

return warped.astype(np.uint8), mask

def image_to_cylindrical_vectorized(img, focal_length):
    """
    Vectorized version of cylindrical projection (much faster).
    """

```



```

h, w = img.shape[:2]

# Center of image
cx = w / 2.0
cy = h / 2.0

# Create coordinate grids
y_cyl, x_cyl = np.mgrid[0:h, 0:w].astype(np.float32)

# Convert to angles
theta = (x_cyl - cx) / focal_length

# Convert to planar coordinates
x_planar = focal_length * np.tan(theta) + cx
y_planar = (y_cyl - cy) / np.cos(theta) + cy

# Create mask for valid coordinates
mask = (x_planar >= 0) & (x_planar < w-1) & (y_planar >= 0) & (y_planar <
↪h-1)

# Initialize output
if len(img.shape) == 3:
    warped = np.zeros((h, w, 3), dtype=np.float32)
else:
    warped = np.zeros((h, w), dtype=np.float32)

# Bilinear interpolation
x0 = np.floor(x_planar[mask]).astype(int)
x1 = x0 + 1
y0 = np.floor(y_planar[mask]).astype(int)
y1 = y0 + 1

dx = x_planar[mask] - x0
dy = y_planar[mask] - y0

if len(img.shape) == 3:
    for c in range(3):
        top = (1 - dx) * img[y0, x0, c] + dx * img[y0, x1, c]
        bottom = (1 - dx) * img[y1, x0, c] + dx * img[y1, x1, c]
        warped[mask, c] = (1 - dy) * top + dy * bottom
else:
    top = (1 - dx) * img[y0, x0] + dx * img[y0, x1]
    bottom = (1 - dx) * img[y1, x0] + dx * img[y1, x1]
    warped[mask] = (1 - dy) * top + dy * bottom

return warped.astype(np.uint8), mask

```

```

[50]: # =====
# SPHERICAL PROJECTION
# =====

def image_to_spherical(img, focal_length):
    """
    Convert image from planar to spherical projection.

    Projects the image onto a sphere. Works for full 360° panoramas
    and can handle vertical field of view as well.

    Parameters:
    -----
    img : input image (planar projection)
    focal_length : focal length in pixels

    Returns:
    -----
    warped : spherically projected image
    mask : valid pixel mask

    Concept:
    -----
    For each pixel (x_sph, y_sph) in spherical image:
        = (x_sph - center_x) / focal_length (longitude)
        = (y_sph - center_y) / focal_length (latitude)

        x_planar = focal_length * tan() / cos() + center_x
        y_planar = focal_length * tan() + center_y
    """
    h, w = img.shape[:2]

    cx = w / 2.0
    cy = h / 2.0

    # Create coordinate grids
    y_sph, x_sph = np.mgrid[0:h, 0:w].astype(np.float32)

    # Convert to spherical angles
    theta = (x_sph - cx) / focal_length # longitude
    phi = (y_sph - cy) / focal_length # latitude

    # Convert to planar coordinates
    x_planar = focal_length * np.tan(theta) / np.cos(phi) + cx
    y_planar = focal_length * np.tan(phi) + cy

    # Create mask

```

```

    mask = (x_planar >= 0) & (x_planar < w-1) & (y_planar >= 0) & (y_planar <
↪h-1)

    # Initialize output
    if len(img.shape) == 3:
        warped = np.zeros((h, w, 3), dtype=np.float32)
    else:
        warped = np.zeros((h, w), dtype=np.float32)

    # Bilinear interpolation
    x0 = np.floor(x_planar[mask]).astype(int)
    x1 = x0 + 1
    y0 = np.floor(y_planar[mask]).astype(int)
    y1 = y0 + 1

    dx = x_planar[mask] - x0
    dy = y_planar[mask] - y0

    if len(img.shape) == 3:
        for c in range(3):
            top = (1 - dx) * img[y0, x0, c] + dx * img[y0, x1, c]
            bottom = (1 - dx) * img[y1, x0, c] + dx * img[y1, x1, c]
            warped[mask, c] = (1 - dy) * top + dy * bottom
    else:
        top = (1 - dx) * img[y0, x0] + dx * img[y0, x1]
        bottom = (1 - dx) * img[y1, x0] + dx * img[y1, x1]
        warped[mask] = (1 - dy) * top + dy * bottom

    return warped.astype(np.uint8), mask

```

```

[51]: # =====
# AUTOMATIC FOCAL LENGTH ESTIMATION
# =====

def estimate_focal_length(img, fov_degrees=60):
    """
    Estimate focal length from image width and field of view.

    Parameters:
    -----
    img : input image
    fov_degrees : horizontal field of view in degrees (typical: 50-70)

    Returns:
    -----
    focal_length : estimated focal length in pixels
    """

```

Formula:

```
-----  
focal_length = (image_width / 2) / tan(fov / 2)  
"""  
w = img.shape[1]  
fov_rad = np.deg2rad(fov_degrees)  
focal_length = (w / 2.0) / np.tan(fov_rad / 2.0)  
return focal_length
```

```
[57]: # =====  
# CYLINDRICAL PANORAMA PIPELINE  
# =====  
  
def create_cylindrical_panorama(images, homographies, ref_idx=0,   
    ↪focal_length=None):  
    """  
    Create panorama using cylindrical projection.  
  
    Steps:  
    1. Convert all images to cylindrical projection  
    2. Compute homographies between cylindrical images  
    3. Stitch with blending  
  
    Parameters:  
    -----  
    images : list of planar images  
    homographies : list of homographies (or None to recompute)  
    ref_idx : reference image index  
    focal_length : focal length for projection (None = auto-estimate)  
  
    Returns:  
    -----  
    panorama : stitched cylindrical panorama  
    """  
    print("=" * 70)  
    print("CYLINDRICAL PANORAMA CREATION")  
    print("=" * 70)  
  
    # Estimate focal length if not provided  
    if focal_length is None:  
        focal_length = estimate_focal_length(images[0], fov_degrees=60)  
        print(f"Estimated focal length: {focal_length:.1f} pixels")  
    else:  
        print(f"Using focal length: {focal_length:.1f} pixels")  
  
    # Step 1: Convert all images to cylindrical projection  
    print("\nStep 1: Converting images to cylindrical projection...")
```

```

cylindrical_images = []
masks = []

for i, img in enumerate(images):
    print(f" Processing image {i+1}/{len(images)}...")
    cyl_img, mask = image_to_cylindrical_vectorized(img, focal_length)
    cylindrical_images.append(cyl_img)
    masks.append(mask)

# Step 2: Create mosaic (homographies should work better now!)
print("\nStep 2: Creating mosaic from cylindrical images...")

# Use existing homographies or identity (cylindrical images are easier to
↪align)
if homographies is None:
    homographies = [np.eye(3) for _ in images]

# Create mosaic with Laplacian pyramid blending
mosaic = create_mosaic_laplacian(cylindrical_images, homographies,
                                ref_idx, levels=2)

print("\n Cylindrical panorama complete!")
return mosaic, cylindrical_images

```

```

[58]: # =====
# VISUALIZATION AND COMPARISON
# =====

def compare_projections(img, focal_lengths=[None, 'auto', 500, 800, 1200]):
    """
    Compare different projection types and focal lengths.
    """
    print("Comparing different projections and focal lengths...")

    # Auto-estimate if needed
    if 'auto' in focal_lengths:
        auto_f = estimate_focal_length(img, fov_degrees=60)
        focal_lengths = [f if f != 'auto' else auto_f for f in focal_lengths]

    n_projections = 2 # Planar, Cylindrical, Spherical
    n_focal = len([f for f in focal_lengths if f is not None])

    fig, axes = plt.subplots(n_projections + 1, max(3, n_focal),
                             figsize=(18, 12))

    # Original planar
    axes[0, 0].imshow(img)

```

```

    axes[0, 0].set_title('Original\n(Planar Projection)', fontsize=11,
↪fontweight='bold')
    axes[0, 0].axis('off')

    for j in range(1, axes.shape[1]):
        axes[0, j].axis('off')

    # Cylindrical projections with different focal lengths
    row = 1
    for j, f in enumerate([f for f in focal_lengths if f is not None]):
        cyl_img, mask = image_to_cylindrical_vectorized(img, f)
        axes[row, j].imshow(cyl_img)
        axes[row, j].set_title(f'Cylindrical\nf={f:.0f}px', fontsize=10)
        axes[row, j].axis('off')

    # Spherical projections
    row = 2
    for j, f in enumerate([f for f in focal_lengths if f is not None]):
        sph_img, mask = image_to_spherical(img, f)
        axes[row, j].imshow(sph_img)
        axes[row, j].set_title(f'Spherical\nf={f:.0f}px', fontsize=10)
        axes[row, j].axis('off')

plt.tight_layout()
plt.show()

print("\nKey observations:")
print(" • Shorter focal length = more curvature (wider angle)")
print(" • Longer focal length = less curvature (narrower angle)")
print(" • Cylindrical: preserves vertical lines")
print(" • Spherical: curves both horizontally and vertically")

def visualize_projection_concept():
    """
    Create a visual explanation of how projections work.
    """
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))

    # Planar projection
    ax = axes[0]
    ax.text(0.5, 0.9, 'PLANAR PROJECTION', ha='center', fontsize=14,
↪fontweight='bold')
    ax.text(0.5, 0.7, ' Camera', ha='center', fontsize=12)
    ax.plot([0.5], [0.6], 'ko', markersize=10)
    ax.plot([0.2, 0.8], [0.3, 0.3], 'b-', linewidth=3)
    ax.text(0.5, 0.2, 'Flat image plane', ha='center', fontsize=10)

```

```

ax.text(0.5, 0.05, 'Good for: < 90° FOV\nProblem: Distortion at edges',
        ha='center', fontsize=9, style='italic')
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis('off')

# Cylindrical projection
ax = axes[1]
ax.text(0.5, 0.9, 'CYLINDRICAL PROJECTION', ha='center', fontsize=14,
↪fontweight='bold')
ax.text(0.5, 0.7, ' Camera', ha='center', fontsize=12)
ax.plot([0.5], [0.6], 'ko', markersize=10)
theta = np.linspace(-np.pi/3, np.pi/3, 50)
x = 0.5 + 0.3 * np.sin(theta)
y = 0.35 + 0.05 * np.cos(theta)
ax.plot(x, y, 'b-', linewidth=3)
ax.text(0.5, 0.2, 'Cylindrical surface', ha='center', fontsize=10)
ax.text(0.5, 0.05, 'Good for: 180-360° horizontal\nPreserves: Vertical
↪lines',
        ha='center', fontsize=9, style='italic')
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis('off')

# Spherical projection
ax = axes[2]
ax.text(0.5, 0.9, 'SPHERICAL PROJECTION', ha='center', fontsize=14,
↪fontweight='bold')
ax.text(0.5, 0.7, ' Camera', ha='center', fontsize=12)
ax.plot([0.5], [0.6], 'ko', markersize=10)
theta = np.linspace(0, 2*np.pi, 50)
x = 0.5 + 0.15 * np.cos(theta)
y = 0.35 + 0.15 * np.sin(theta)
ax.plot(x, y, 'b-', linewidth=3)
ax.text(0.5, 0.1, 'Spherical surface', ha='center', fontsize=10)
ax.text(0.5, 0.02, 'Good for: Full 360° × 180°\nBest for: Immersive
↪panoramas',
        ha='center', fontsize=9, style='italic')
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis('off')

plt.tight_layout()
plt.show()

```

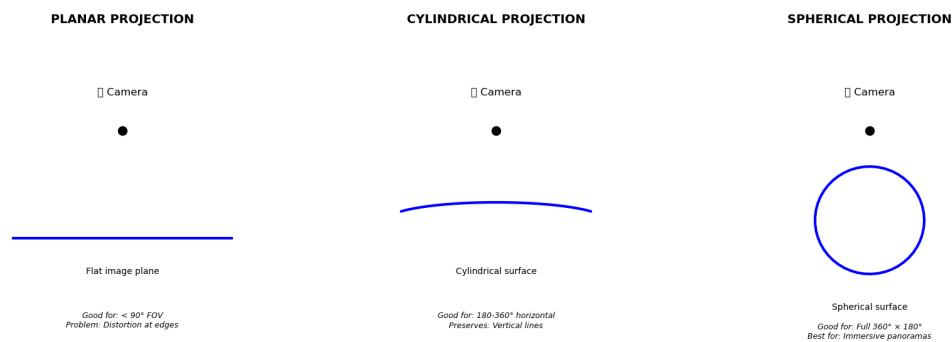
```
[59]: # =====
# PRACTICAL GUIDE: CYLINDRICAL AND SPHERICAL PROJECTIONS
# =====

print("=" * 70)
print("UNDERSTANDING PROJECTIONS FOR WIDE PANORAMAS")
print("=" * 70)

# First, visualize the concept
visualize_projection_concept()
```

```
=====
UNDERSTANDING PROJECTIONS FOR WIDE PANORAMAS
=====
```

```
C:\Users\jorda\AppData\Local\Temp\ipykernel_37380\309868852.py:107: UserWarning:
Glyph 128247 (\N{CAMERA}) missing from font(s) DejaVu Sans.
plt.tight_layout()
```



```
[55]: # =====
# EXAMPLE 1: SINGLE IMAGE PROJECTION COMPARISON
# =====

print("\n" + "=" * 70)
print("EXAMPLE 1: Testing Projections on a Single Image")
print("=" * 70)

# Load one of your images
img_test = img1 # Or use any image you have

# Compare different projections and focal lengths
compare_projections(img_test, focal_lengths=['auto', 500, 800, 1200])

print("\n Key Insight:")
```



```
print("Before stitching, convert each image to cylindrical/spherical projection.
↪")
print("This 'pre-warps' them so they align better for wide panoramas.")
```

EXAMPLE 1: Testing Projections on a Single Image

Comparing different projections and focal lengths...



Key observations:

- Shorter focal length = more curvature (wider angle)
- Longer focal length = less curvature (narrower angle)
- Cylindrical: preserves vertical lines
- Spherical: curves both horizontally and vertically

Key Insight:

Before stitching, convert each image to cylindrical/spherical projection.
This 'pre-warps' them so they align better for wide panoramas.

```
[60]: # =====
# EXAMPLE 2: CREATING CYLINDRICAL PANORAMA (2 IMAGES)
# =====
```

```

print("\n" + "=" * 70)
print("EXAMPLE 2: Creating Cylindrical Panorama from Your Images")
print("=" * 70)

# Method 1: Automatic focal length
print("\n Method 1: Auto-estimate focal length")
panorama_cyl_auto, cyl_images_auto = create_cylindrical_panorama(
    images=[img1, img2],
    homographies=[np.eye(3), H_2to1], # Your existing homography
    ref_idx=0,
    focal_length=None # Auto-estimate
)

# Display
fig, axes = plt.subplots(2, 2, figsize=(18, 12))

axes[0, 0].imshow(img1)
axes[0, 0].set_title('Original Image 1\n(Planar)', fontsize=12)
axes[0, 0].axis('off')

axes[0, 1].imshow(img2)
axes[0, 1].set_title('Original Image 2\n(Planar)', fontsize=12)
axes[0, 1].axis('off')

axes[1, 0].imshow(cyl_images_auto[0])
axes[1, 0].set_title('Cylindrical Image 1\n(Pre-warped)', fontsize=12)
axes[1, 0].axis('off')

axes[1, 1].imshow(cyl_images_auto[1])
axes[1, 1].set_title('Cylindrical Image 2\n(Pre-warped)', fontsize=12)
axes[1, 1].axis('off')

plt.tight_layout()
plt.show()

# Show the final panorama
plt.figure(figsize=(20, 8))
plt.imshow(panorama_cyl_auto)
plt.title('Cylindrical Panorama (Auto Focal Length)', fontsize=14,
        fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig('cylindrical_panorama_auto.jpg', dpi=150, bbox_inches='tight')
plt.show()

```

=====

EXAMPLE 2: Creating Cylindrical Panorama from Your Images

=====
Method 1: Auto-estimate focal length

=====
CYLINDRICAL PANORAMA CREATION
=====

Estimated focal length: 2025.6 pixels

Step 1: Converting images to cylindrical projection...

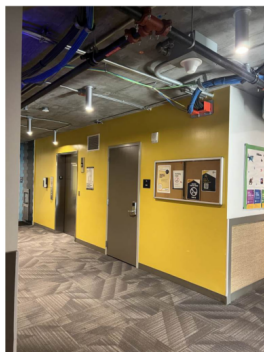
Processing image 1/2...

Processing image 2/2...

Step 2: Creating mosaic from cylindrical images...

Cylindrical panorama complete!

Original Image 1
(Planar)



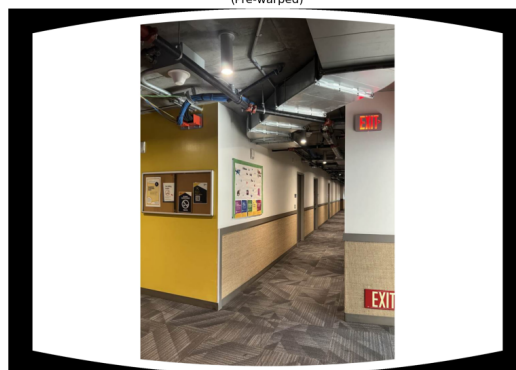
Original Image 2
(Planar)



Cylindrical Image 1
(Pre-warped)



Cylindrical Image 2
(Pre-warped)



Cylindrical Panorama (Auto Focal Length)



```
[62]: # =====  
# EXAMPLE 3: COMPARE PLANAR vs CYLINDRICAL  
# =====  
  
print("\n" + "=" * 70)  
print("EXAMPLE 3: Comparing Planar vs Cylindrical Panoramas")  
print("=" * 70)  
  
# Create planar panorama (your original method)  
print("Creating planar panorama...")  
mosaic_planar = create_mosaic_laplacian(  
    [img1, img2], [np.eye(3), H_2to1], ref_idx=0, levels=2  
)  
  
# Create cylindrical panorama with different focal lengths  
print("\nCreating cylindrical panoramas with different focal lengths...")  
  
focal_lengths = {  
    'Short (wide angle)': 500,  
    'Medium': 800,  
    'Long (narrow angle)': 1200,  
    'Auto-estimated': estimate_focal_length(img1, fov_degrees=60)  
}
```

```

cylindrical_results = {}
for name, f in focal_lengths.items():
    print(f"  Testing {name} (f={f:.0f})...")
    pano, cyl_imgs = create_cylindrical_panorama(
        [img1, img2], [np.eye(3), H_2to1], ref_idx=0, focal_length=f
    )
    cylindrical_results[name] = pano

# Visualize comparison
fig, axes = plt.subplots(3, 2, figsize=(18, 14))

# Planar
axes[0, 0].imshow(mosaic_planar)
axes[0, 0].set_title('PLANAR PROJECTION\n(Original method)',
                    fontsize=12, fontweight='bold')
axes[0, 0].axis('off')

# Zoom on planar edges
h, w = mosaic_planar.shape[:2]
axes[0, 1].imshow(mosaic_planar[:, :w//3]) # Left edge
axes[0, 1].set_title('Planar - Left Edge\n(notice distortion?)', fontsize=10)
axes[0, 1].axis('off')

# Cylindrical results
row = 1
col = 0
for name, pano in list(cylindrical_results.items())[:4]:
    axes[row, col].imshow(pano)
    axes[row, col].set_title(f'CYLINDRICAL - {name}', fontsize=11)
    axes[row, col].axis('off')

    col += 1
    if col > 1:
        col = 0
        row += 1

plt.suptitle('Planar vs Cylindrical Projection Comparison',
            fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

```

=====

EXAMPLE 3: Comparing Planar vs Cylindrical Panoramas

=====

Creating planar panorama...

Creating cylindrical panoramas with different focal lengths...

Testing Short (wide angle) (f=500)...

=====

CYLINDRICAL PANORAMA CREATION

=====

Using focal length: 500.0 pixels

Step 1: Converting images to cylindrical projection...

Processing image 1/2...

Processing image 2/2...

Step 2: Creating mosaic from cylindrical images...

Cylindrical panorama complete!

Testing Medium (f=800)...

=====

CYLINDRICAL PANORAMA CREATION

=====

Using focal length: 800.0 pixels

Step 1: Converting images to cylindrical projection...

Processing image 1/2...

Processing image 2/2...

Step 2: Creating mosaic from cylindrical images...

Cylindrical panorama complete!

Testing Long (narrow angle) (f=1200)...

=====

CYLINDRICAL PANORAMA CREATION

=====

Using focal length: 1200.0 pixels

Step 1: Converting images to cylindrical projection...

Processing image 1/2...

Processing image 2/2...

Step 2: Creating mosaic from cylindrical images...

Cylindrical panorama complete!

Testing Auto-estimated (f=2026)...

=====

CYLINDRICAL PANORAMA CREATION

=====

Using focal length: 2025.6 pixels

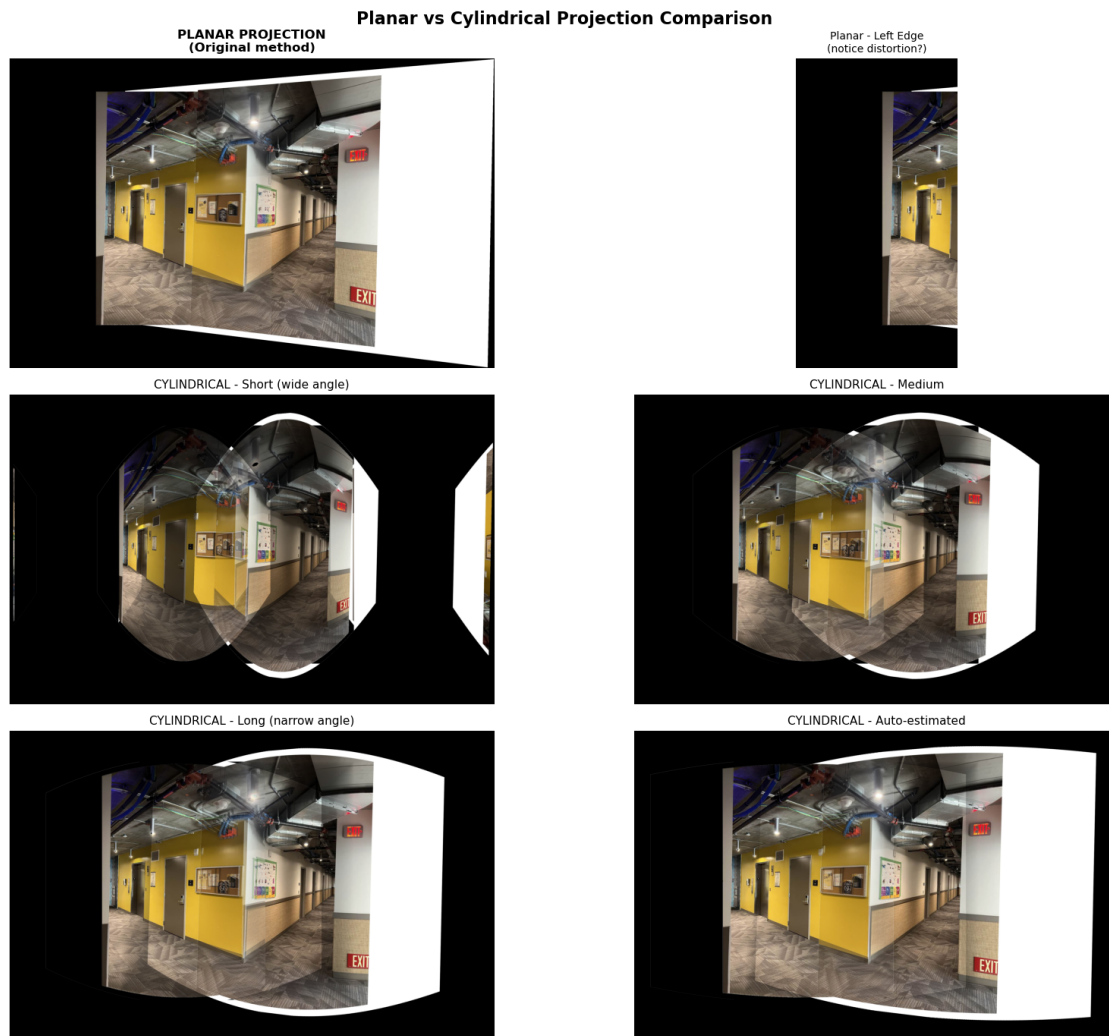
Step 1: Converting images to cylindrical projection...

Processing image 1/2...

Processing image 2/2...

Step 2: Creating mosaic from cylindrical images...

Cylindrical panorama complete!



```
[63]: # =====  
# EXAMPLE 4: CHOOSING THE RIGHT FOCAL LENGTH  
# =====  
  
print("\n" + "=" * 70)  
print("EXAMPLE 4: How to Choose Focal Length")  
print("=" * 70)  
  
def analyze_focal_length_effect(img):
```

```

"""
Show how different focal lengths affect the projection.
"""

focal_lengths = [400, 600, 800, 1000, 1200]

fig, axes = plt.subplots(2, len(focal_lengths), figsize=(20, 8))

for i, f in enumerate(focal_lengths):
    # Cylindrical
    cyl, mask = image_to_cylindrical_vectorized(img, f)
    axes[0, i].imshow(cyl)
    axes[0, i].set_title(f'f={f}px\n(Cylindrical)', fontsize=10)
    axes[0, i].axis('off')

    # Spherical
    sph, mask = image_to_spherical(img, f)
    axes[1, i].imshow(sph)
    axes[1, i].set_title(f'f={f}px\n(Spherical)', fontsize=10)
    axes[1, i].axis('off')

fig.suptitle('Effect of Focal Length on Projection\n(Shorter = More ↵
↳Curved)',
            fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n Focal Length Guidelines:")
print(" • f = 400-600: Wide angle (> 90° FOV) - high curvature")
print(" • f = 600-900: Normal (50-70° FOV) - moderate curvature")
print(" • f = 900-1200: Narrow (< 50° FOV) - minimal curvature")
print(" • Rule of thumb: f  image_width / 2 for 60° FOV")

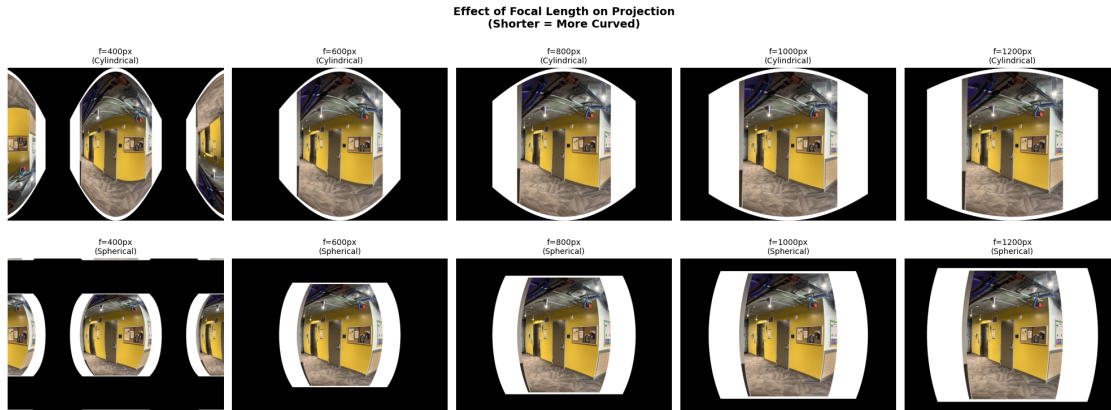
analyze_focal_length_effect(img1)

```

=====

EXAMPLE 4: How to Choose Focal Length

=====



Focal Length Guidelines:

- $f = 400\text{--}600$: Wide angle ($> 90^\circ$ FOV) - high curvature
- $f = 600\text{--}900$: Normal ($50\text{--}70^\circ$ FOV) - moderate curvature
- $f = 900\text{--}1200$: Narrow ($< 50^\circ$ FOV) - minimal curvature
- Rule of thumb: $f = \text{image_width} / 2$ for 60° FOV

```
[64]: # =====
# EXAMPLE 5: WHEN TO USE EACH PROJECTION
# =====

print("\n" + "=" * 70)
print("DECISION GUIDE: Which Projection Should You Use?")
print("=" * 70)

decision_guide = """

    PANORAMA WIDTH      RECOMMENDED PROJECTION      WHY?

    < 90°                PLANAR (Homography)          Simple, accurate
                                                                No distortion

    90° - 180°           CYLINDRICAL                  Reduces edge
                                                                distortion
                                                                Preserves vertical

    180° - 360°          CYLINDRICAL                  Natural for
                                                                horizontal pano

    360° × 180°          SPHERICAL                    Full immersive
    (Full sphere)                                     panorama

```

ADDITIONAL FACTORS:

Use CYLINDRICAL when:

- Panorama is mostly horizontal rotation
- Want to preserve vertical lines (buildings, trees)
- Field of view is 90° - 360° horizontally

Use SPHERICAL when:

- Camera rotates both horizontally AND vertically
- Creating VR/360° content
- Field of view is very wide in all directions

Use PLANAR when:

- Panorama is narrow (< 90°)
- Images have good overlap
- Speed is important (simplest method)

"""

```
print(decision_guide)
```

=====

DECISION GUIDE: Which Projection Should You Use?

=====

PANORAMA WIDTH	RECOMMENDED PROJECTION	WHY?
< 90°	PLANAR (Homography)	Simple, accurate No distortion
90° - 180°	CYLINDRICAL	Reduces edge distortion Preserves vertical
180° - 360°	CYLINDRICAL	Natural for horizontal pano
360° × 180° (Full sphere)	SPHERICAL	Full immersive panorama

ADDITIONAL FACTORS:

Use CYLINDRICAL when:

- Panorama is mostly horizontal rotation
- Want to preserve vertical lines (buildings, trees)
- Field of view is 90° - 360° horizontally

Use SPHERICAL when:

- Camera rotates both horizontally AND vertically
- Creating VR/360° content
- Field of view is very wide in all directions

Use PLANAR when:

- Panorama is narrow ($< 90^\circ$)
- Images have good overlap
- Speed is important (simplest method)

[]:

```
[76]: # =====
# EXAMPLE 6: COMPLETE WORKFLOW WITH CYLINDRICAL
# =====

def create_cylindrical_pano_complete(images, ref_idx=1, fov_estimate=60):
    """
    Complete pipeline for cylindrical panorama.

    This is what you'd use for your assignment when panorama > 120°!

    Parameters:
    -----
    images : list of images [img1, img2, img3, ...]
    ref_idx : index of reference image (usually center)
    fov_estimate : estimated field of view in degrees (50-70 typical)

    Returns:
    -----
    panorama : final cylindrical panorama
    cyl_images : list of cylindrically projected images (for debugging)
    homographies : list of computed homographies
    """
    print("\n CYLINDRICAL PANORAMA PIPELINE")
    print("-" * 50)

    # Step 1: Estimate focal length
    f = estimate_focal_length(images[ref_idx], fov_degrees=fov_estimate)
    print(f"Step 1: Estimated focal length = {f:.1f} pixels")
    print(f"          (based on {fov_estimate}° FOV)")

    # Step 2: Convert to cylindrical
    print("\nStep 2: Converting images to cylindrical projection...")
    cyl_images = []
```

```

for i, img in enumerate(images):
    cyl_img, _ = image_to_cylindrical_vectorized(img, f)
    cyl_images.append(cyl_img)
    print(f"        Image {i+1} converted ")

# Display cylindrical images
print("\nDisplaying cylindrical projections...")
n_imgs = len(cyl_images)
fig, axes = plt.subplots(2, n_imgs, figsize=(18, 8))

for i in range(n_imgs):
    # Original
    if n_imgs > 1:
        axes[0, i].imshow(images[i])
        axes[0, i].set_title(f'Original {i+1}')
        axes[0, i].axis('off')
        # Cylindrical
        axes[1, i].imshow(cyl_images[i])
        axes[1, i].set_title(f'Cylindrical {i+1}')
        axes[1, i].axis('off')
    else:
        axes[0].imshow(images[i])
        axes[0].set_title(f'Original {i+1}')
        axes[0].axis('off')
        axes[1].imshow(cyl_images[i])
        axes[1].set_title(f'Cylindrical {i+1}')
        axes[1].axis('off')

plt.tight_layout()
plt.show()

# Step 3: Select points and compute homographies
print("\nStep 3: Computing homographies between cylindrical images...")
print("        NOTE: Homographies work MUCH better on cylindrical images!")

homographies = []

for i in range(len(cyl_images)):
    if i == ref_idx:
        # Reference image gets identity
        homographies.append(np.eye(3))
        print(f"        Image {i+1}: Reference (identity)")
    else:
        # Need to compute homography to reference
        if i < ref_idx:
            # Left side images → warp to reference

```

```

        print(f"\n          Selecting points: Image {i+1} → Image_{ref_idx+1}")
        points_src, points_ref = select_correspondences_cylindrical(
            cyl_images[i], cyl_images[ref_idx], i+1, ref_idx+1
        )
    else:
        # Right side images → warp to reference
        print(f"\n          Selecting points: Image {i+1} → Image_{ref_idx+1}")
        points_src, points_ref = select_correspondences_cylindrical(
            cyl_images[i], cyl_images[ref_idx], i+1, ref_idx+1
        )

    if points_src is None or points_ref is None:
        print("          Point selection failed, using identity")
        homographies.append(np.eye(3))
    else:
        H = computeH_with_details(points_src, points_ref)
        homographies.append(H)
        print(f"          Homography computed ")

        # Test reprojection error
        homogeneous_pts = np.column_stack([points_src, np.
        ↪ones(len(points_src))])
        transformed = (H @ homogeneous_pts.T).T
        transformed = transformed[:, :2] / transformed[:, 2:3]
        errors = np.linalg.norm(points_ref - transformed, axis=1)
        print(f"          Mean error: {np.mean(errors):.2f} pixels")

    # Step 4: Create cylindrical mosaic
    print("\nStep 4: Creating cylindrical panorama with Laplacian blending...")
    panorama = create_mosaic_laplacian(cyl_images, homographies, ref_idx,
    ↪levels=4)

    # Display result
    plt.figure(figsize=(20, 8))
    plt.imshow(panorama)
    plt.title('Cylindrical Panorama (Laplacian Pyramid Blending)',
              fontsize=14, fontweight='bold')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

    print("\n Cylindrical panorama complete!")
    print(f"    Final size: {panorama.shape[1]} x {panorama.shape[0]} pixels")

    return panorama, cyl_images, homographies

```

```

def select_correspondences_cylindrical(img1, img2, img1_num, img2_num):
    """
    Helper function for selecting correspondences between cylindrical images.
    """
    %matplotlib qt

    # Select points on first image
    fig = plt.figure(figsize=(12, 8))
    plt.imshow(img1)
    plt.title(f'Cylindrical Image {img1_num}: Click points (right-click when_
done)')
    plt.axis('off')
    points1_list = plt.ginput(6, timeout=0)
    plt.close()

    # Select points on second image
    fig = plt.figure(figsize=(12, 8))
    plt.imshow(img2)
    plt.title(f'Cylindrical Image {img2_num}: Click SAME points in SAME order')
    plt.axis('off')
    points2_list = plt.ginput(6, timeout=0)
    plt.close()

    %matplotlib inline

    if len(points1_list) != len(points2_list):
        print(f" Mismatch: {len(points1_list)} vs {len(points2_list)} points")
        return None, None

    if len(points1_list) < 4:
        print(f" Need at least 4 points, got {len(points1_list)}")
        return None, None

    points1 = np.array(points1_list)
    points2 = np.array(points2_list)

    # Visualize
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
    ax1.imshow(img1)
    ax1.set_title(f'Cylindrical Image {img1_num}')
    ax1.axis('off')
    for i, (x, y) in enumerate(points1):
        ax1.plot(x, y, 'ro', markersize=8)
        ax1.text(x, y, str(i+1), color='white', fontsize=10, fontweight='bold',

```

```

        bbox=dict(boxstyle="round,pad=0.3", facecolor='red', alpha=0.
↪7))

    ax2.imshow(img2)
    ax2.set_title(f'Cylindrical Image {img2_num}')
    ax2.axis('off')
    for i, (x, y) in enumerate(points2):
        ax2.plot(x, y, 'ro', markersize=8)
        ax2.text(x, y, str(i+1), color='white', fontsize=10, fontweight='bold',
                bbox=dict(boxstyle="round,pad=0.3", facecolor='red', alpha=0.
↪7))

    plt.tight_layout()
    plt.show()

    return points1, points2

```

```

[ ]: # =====
# USAGE EXAMPLES
# =====

print("\n" + "=" * 70)
print("CYLINDRICAL PANORAMA EXAMPLES")
print("=" * 70)

# Example 1: Three images with center as reference
print("\nExample: Creating cylindrical panorama from 3 images")
print("-" * 50)
img5 =
img6 =
img7 =
# Assuming you have img5, img6, img7 loaded
images_cyl = [img5, img6, img7]

panorama_cyl, cyl_imgs, H_list = create_cylindrical_pano_complete(
    images_cyl,
    ref_idx=1,          # img6 (center) is reference
    fov_estimate=60     # Adjust based on your camera
)

# Save result
cv2.imwrite('panorama_cylindrical.jpg', cv2.cvtColor(panorama_cyl, cv2.
↪COLOR_RGB2BGR))
print("\nSaved panorama_cylindrical.jpg")

# =====

```

```

# COMPARE PLANAR vs CYLINDRICAL
# =====

print("\n" + "=" * 70)
print("COMPARISON: Planar vs Cylindrical Projection")
print("=" * 70)

# You already created a planar mosaic with the same images
# Let's compare them side by side

fig, axes = plt.subplots(2, 1, figsize=(20, 12))

axes[0].imshow(mosaic_lap_3) # Your planar 3-image mosaic
axes[0].set_title('PLANAR Projection Mosaic', fontsize=14, fontweight='bold')
axes[0].axis('off')

axes[1].imshow(panorama_cyl)
axes[1].set_title('CYLINDRICAL Projection Mosaic', fontsize=14,
    ↪fontweight='bold', color='green')
axes[1].axis('off')

plt.tight_layout()
plt.savefig('planar_vs_cylindrical_comparison.jpg', dpi=150,
    ↪bbox_inches='tight')
plt.show()

print("\n Comparison:")
print("-" * 50)
print("PLANAR advantages:")
print("    Simpler (no extra warping step)")
print("    Good for narrow FOV (< 90°)")
print("    Preserves original image quality in center")
print("\nCYLINDRICAL advantages:")
print("    Better for wide panoramas (90° - 360°)")
print("    Reduces distortion at edges")
print("    Preserves vertical lines (buildings, trees)")
print("    More natural appearance for wide scenes")

print("\n Rule of thumb:")
print("    • FOV < 90°: Use PLANAR")
print("    • FOV 90-180°: Use CYLINDRICAL")
print("    • FOV > 180°: Use CYLINDRICAL or SPHERICAL")

```

[]: